

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN
NGÀNH: MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU



BÁO CÁO ĐỒ ÁN CUỐI KỲ
THIẾT KẾ VÀ TRIỂN KHAI MẠNG
METRO ETHERNET SỬ DỤNG MPLS
CHO KẾT NỐI ĐA CHI NHÁNH DOANH NGHIỆP

Giảng viên hướng dẫn: ThS. Lê Viết Thanh
Sinh viên thực hiện: Huỳnh Văn Dũng
Mã số sinh viên: 52300190
Nhóm/Lớp: 26_TKM

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025–2026

LỜI CẢM ƠN

Để hoàn thành báo cáo đồ án cuối kỳ này, em xin gửi lời cảm ơn chân thành đến:

- Em xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Tôn Đức Thắng và Khoa Công nghệ Thông tin đã đưa môn **Thiết kế Mạng** vào chương trình đào tạo ngành Mạng máy tính và Truyền thông dữ liệu, đồng thời tạo điều kiện về cơ sở vật chất và môi trường học tập thuận lợi.
- Em đặc biệt biết ơn **ThS. Lê Viết Thanh** – giảng viên hướng dẫn môn học – vì sự hướng dẫn tận tâm, truyền đạt những kiến thức chuyên môn quý báu về kiến trúc mạng, công nghệ MPLS và Metro Ethernet, giúp em có nền tảng vững chắc để thực hiện đề tài này.
- Đề tài “*Thiết kế và Triển khai Mạng Metro Ethernet sử dụng MPLS cho Kết nối Đa Chi nhánh Doanh nghiệp*” đòi hỏi sự tổng hợp kiến thức từ nhiều lĩnh vực: lý thuyết mạng, Linux networking, và công cụ mô phỏng Mininet. Do kiến thức và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được ý kiến đóng góp của thầy để hoàn thiện hơn.

Lời cuối cùng, em xin kính chúc thầy nhiều sức khỏe, thành công trên con đường giảng dạy và hạnh phúc trong cuộc sống.

TP. Hồ Chí Minh, tháng 6 năm 2026

Sinh viên thực hiện

(Ký tên và ghi rõ họ tên)

Huỳnh Văn Dũng

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **ThS. Lê Việt Thanh**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ môi trường mô phỏng Mininet và các nguồn tài liệu được ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu kỹ thuật của các tổ chức chuẩn hóa (IETF, MEF) và nhà sản xuất thiết bị (Cisco, Arista) đều có trích dẫn và chú thích nguồn gốc rõ ràng.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung báo cáo của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, tháng 6 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Huỳnh Văn Dũng

TÓM TẮT

Đề tài trình bày việc **thiết kế và triển khai mô hình mạng Metro Ethernet** sử dụng công nghệ **MPLS (Multiprotocol Label Switching)** nhằm kết nối ba chi nhánh doanh nghiệp thông qua hạ tầng của nhà cung cấp dịch vụ (ISP). Toàn bộ hệ thống được mô phỏng trên nền tảng **Mininet** và **Ubuntu 22.04**, sử dụng Linux Kernel MPLS (iproute2).

Mục tiêu: Nghiên cứu và so sánh hiệu năng mạng của ba kiến trúc LAN khác nhau tại mỗi chi nhánh – Mạng phẳng (Flat Network), Mạng 3 lớp truyền thống (Core–Distribution–Access), và Mạng Leaf-Spine – khi hoạt động qua cùng một backbone MPLS.

Phương pháp: Sử dụng Mininet để mô phỏng topology đầy đủ bao gồm các router CE (Customer Edge), PE (Provider Edge 3 node), P (Provider Core 4 node). Cơ chế chuyển mạch nhãn MPLS (Push/Swap/Pop) được triển khai thông qua Linux Kernel module `mpls_router` và `iproute2`. Đo lường hiệu năng tự động bằng `iperf3` (Throughput UDP/TCP, Jitter, Packet Loss) và `ping` (Delay/RTT), lặp lại 3 lần mỗi kịch bản (42 record tổng cộng).

Kết quả chính:

- **Backbone MPLS** hoạt động ổn định: packet loss $\approx 0\%$, delay cross-branch $\approx 0.10\text{--}0.115\text{ ms}$, throughput đạt $\approx 99.98\text{--}99.99\%$ mức tải UDP.
- **Flat Network:** delay nội bộ 0.052 ms , đơn giản nhất nhưng tiềm ẩn rủi ro broadcast storm khi mở rộng.
- **3-Tier CDA:** delay inter-VLAN $0.076\text{--}0.085\text{ ms}$, hỗ trợ phân vùng VLAN và bảo mật ACL linh hoạt.
- **Leaf-Spine:** jitter nội bộ thấp nhất (0.040 ms), kiến trúc full-mesh loại bỏ STP, phù hợp nhất cho ứng dụng real-time và Data Center.
- Kiến trúc LAN nội bộ **không ảnh hưởng đáng kể** đến hiệu năng cross-branch do bottleneck là backbone MPLS.

Từ khóa: Metro Ethernet, MPLS, Label Switching, Push/Swap/Pop, Mininet, Flat Network, 3-Tier CDA, Leaf-Spine, Throughput, Delay, Jitter, Packet Loss, LSP, PE Router, P Router, CE Router.

Mục lục

LỜI CẢM ƠN	1
CAM KẾT CÔNG TRÌNH	2
TÓM TẮT	3
Chương 1. Giới thiệu	1
1. Bối cảnh và động lực nghiên cứu	1
2. Mục tiêu nghiên cứu	1
3. Phạm vi và giới hạn	1
4. Cấu trúc báo cáo	2
Chương 2. Cơ sở lý thuyết	3
1. Kiến trúc mạng nội bộ (LAN Architectures)	3
1.1. Mạng phẳng (Flat Network)	3
1.2. Mạng 3 lớp truyền thống (Core – Distribution – Access)	3
1.3. Mạng 2 lớp Leaf-Spine	4
2. Công nghệ MPLS (Multiprotocol Label Switching)	4
2.1. Khái niệm và nguyên lý	4
2.2. Cơ chế Push – Swap – Pop	5
2.3. So sánh MPLS với IP Routing truyền thống	5
3. Metro Ethernet và VPLS	5
3.1. Metro Ethernet	5
3.2. VPLS (Virtual Private LAN Service)	6
4. Các giao thức điều khiển MPLS	6
4.1. OSPF (Open Shortest Path First)	6
4.2. LDP (Label Distribution Protocol)	6
4.3. MP-BGP (Multiprotocol BGP)	6
5. Các chỉ số hiệu năng mạng	7
Chương 3. Thiết kế mô hình mạng	8
1. Tổng quan thiết kế	8
1.1. Sơ đồ logic tổng thể	9

2.	Thiết kế chi nhánh 1: Mạng phẳng (Flat Network)	10
2.1.	Cấu trúc topology	10
2.2.	Thông số kỹ thuật	10
3.	Thiết kế chi nhánh 2: Mạng 3 lớp (Core–Distribution–Access)	10
3.1.	Cấu trúc topology	10
3.2.	Thông số kỹ thuật	11
4.	Thiết kế chi nhánh 3: Mạng Leaf-Spine	11
4.1.	Cấu trúc topology	11
4.2.	Thông số kỹ thuật	12
5.	Thiết kế backbone MPLS của ISP	12
5.1.	Topology backbone	12
5.2.	Kế hoạch nhãn MPLS (Label Plan)	13
6.	Bảng địa chỉ IP toàn hệ thống	13
Chương 4.	Triển khai trên Mininet	15
1.	Môi trường cài đặt	15
1.1.	Yêu cầu hệ thống	15
1.2.	Hướng dẫn cài đặt từng bước	15
2.	Cấu trúc file code	16
3.	Mã nguồn chi tiết	17
3.1.	Lớp LinuxRouter và MetroFullTopo	17
3.2.	Xây dựng chi nhánh 1 – Flat Network	18
3.3.	Xây dựng chi nhánh 2 – 3-Tier CDA	19
3.4.	Xây dựng chi nhánh 3 – Leaf-Spine	20
4.	Câu hình MPLS (Push – Swap – Pop)	21
4.1.	Bước 1: Bật MPLS trên các node backbone	21
4.2.	Bước 2: Câu hình PUSH tại PE routers (Ingress LER)	21
4.3.	Bước 3: Câu hình SWAP tại P routers (Transit LSR)	22
4.4.	Xem bảng MPLS route trên router	23
5.	Hướng dẫn chạy topology	23
5.1.	Khởi động topology tổng thể	23
5.2.	Kiểm tra kết nối trong Mininet CLI	24
5.3.	Kết quả ping end-to-end thực tế	24
5.4.	đo lường iperf3 trong CLI	25
6.	Bộ công cụ đo lường tự động	25
6.1.	Tổng quan bộ đo lường	25
Chương 5.	Kết quả đo lường và phân tích	27

1.	Phối hợp pháp đô lũng	27
2.	Kết quả đô lũng nội bộ (Intra-branch)	27
2.1.	Chi nhánh 1 – Flat Network	27
2.2.	Chi nhánh 2 – 3-Tier (Core–Distribution–Access)	28
2.3.	Chi nhánh 3 – Leaf-Spine	28
2.4.	Bảng tổng hợp kết quả nội bộ	29
3.	Kết quả đô lũng xuyên chi nhánh (Cross-branch)	29
3.1.	Log thực tế cross-branch	29
3.2.	Bảng tổng hợp cross-branch	32
4.	Biểu đồ so sánh	32
5.	Phân tích kết quả	36
5.1.	Phân tích độ trễ (Delay)	36
5.2.	Phân tích Throughput và Jitter	36
5.3.	Phân tích Packet Loss	37
6.	đánh giá hoạt động MPLS	37
6.1.	đồng đi gói tin xuyên backbone	37
6.2.	Kiểm tra bảng MPLS route thực tế	37
Chương 6.	Kết luận và hướng phát triển	39
1.	Tóm tắt kết quả đạt được	39
1.1.	Kết luận về hiệu năng đo lường thực tế	40
1.2.	Kết luận về 3 kiến trúc LAN	40
1.3.	Kết luận về MPLS	41
2.	Hướng phát triển	41
3.	Lời kết	42
TÀI LIỆU THAM KHẢO		43

Danh sách hình vẽ

Hình 3.1.	Sơ đồ tổng thể mô hình Metro Ethernet MPLS kết nối 3 chi nhánh	8
Hình 3.2.	Sơ đồ kiến trúc tổng thể hệ thống	9
Hình 5.1.	So sánh độ trễ (Ping RTT) giữa các kiến trúc – intra và cross-branch	32
Hình 5.2.	So sánh Throughput UDP theo mức tải (10/50/100 Mbps)	33
Hình 5.3.	So sánh Jitter giữa các kích bản đồ lõi	33
Hình 5.4.	So sánh độ trễ: nội bộ chi nhánh vs. xuyên backbone MPLS	34
Hình 5.5.	Throughput thực tế vs. mức tải mạng (10, 50, 100 Mbps)	34
Hình 5.6.	Phân phối độ trễ (boxplot) theo từng kiến trúc LAN	35
Hình 5.7.	Tỉ lệ mất gói (Packet Loss) theo từng kích bản	35
Hình 5.8.	Heatmap tổng hợp 4 chỉ số hiệu năng	36

Danh sách bảng

Bảng 2.1.	So sánh MPLS và IP Routing truyền thống	5
Bảng 2.2.	Định nghĩa và đơn vị đo các chỉ số hiệu năng	7
Bảng 3.1.	Bảng phân phối nhãn MPLS (LFIB – Label Forwarding Information Base)	13
Bảng 3.2.	Bảng địa chỉ IP – LAN chi nhánh	13
Bảng 3.3.	Bảng địa chỉ IP – Backbone ISP MPLS	14
Bảng 4.1.	Yêu cầu phần mềm và phần cứng	15
Bảng 5.1.	Kết quả đo lường nội bộ (intra-branch) tại 100 Mbps UDP	29
Bảng 5.2.	Kết quả đo lường cross-branch (xuyên backbone MPLS) tại 100 Mbps UDP	32
Bảng 5.3.	TCP Throughput (Mbps) qua backbone MPLS vs. nội bộ chi nhánh	32
Bảng 6.1.	Tổng hợp kết quả đo lường thực tế – Cross-branch (100 Mbps UDP)	40
Bảng 6.2.	Đánh giá tổng hợp 3 kiến trúc LAN dựa trên kết quả đo lường thực tế	40

CHƯƠNG 1. GIỚI THIỆU

1. Bối cảnh và động lực nghiên cứu

Trong bối cảnh các doanh nghiệp hiện đại ngày càng mở rộng hoạt động ra nhiều chi nhánh ở các địa điểm khác nhau, nhu cầu kết nối hạ tầng mạng giữa các chi nhánh một cách hiệu quả, an toàn và kinh tế đang trở nên cấp thiết hơn bao giờ hết. Các ứng dụng doanh nghiệp như ERP, VoIP, hội nghị truyền hình và chia sẻ dữ liệu nội bộ đặt ra yêu cầu khắt khe về **băng thông, độ trễ thấp và tính liên tục** của dịch vụ.

Công nghệ **Metro Ethernet** (MAN – Metropolitan Area Network dựa trên chuẩn Ethernet) nổi lên như một giải pháp hấp dẫn nhờ khả năng cung cấp kết nối tốc độ cao ở quy mô thành phố với chi phí thấp hơn so với các công nghệ WAN truyền thống (Frame Relay, ATM). Khi kết hợp với **MPLS (Multiprotocol Label Switching)**, Metro Ethernet không chỉ tăng tốc độ chuyển mạch mà còn cho phép định tuyến lưu lượng linh hoạt, hỗ trợ đa dịch vụ và phân tách hoàn toàn lưu lượng của các khách hàng khác nhau trên cùng hạ tầng ISP.

2. Mục tiêu nghiên cứu

Đề tài này nhằm tới các mục tiêu cụ thể sau:

- **Thiết kế** mô hình mạng Metro Ethernet MAN sử dụng MPLS backbone để kết nối ba chi nhánh doanh nghiệp thông qua hạ tầng ISP.
- **Triển khai mô phỏng** toàn bộ mô hình trên Mininet (Ubuntu 22.04), bao gồm ba kiến trúc LAN khác nhau tại mỗi chi nhánh và hệ thống router CE–PE–P của backbone MPLS.
- **Đo lường và so sánh hiệu năng** các chỉ số: Throughput, Delay, Packet Loss và Jitter cho từng kiến trúc LAN khi lưu lượng đi qua backbone MPLS.
- **Phân tích và đánh giá** ưu nhược điểm của ba kiến trúc LAN (Flat, 3-Tier, Leaf-Spine) và cơ chế hoạt động của MPLS so với IP routing truyền thống.

3. Phạm vi và giới hạn

Phạm vi:

- Mô phỏng trên môi trường Mininet (phần mềm) – không phải thiết bị vật lý thực tế.
- Sử dụng Linux kernel MPLS (iproute2) để mô phỏng chuyển mạch nhãn; không dùng phần cứng MPLS chuyên dụng (Cisco, Juniper).
- Ba chi nhánh đại diện cho ba kiến trúc LAN điển hình được nghiên cứu trong học

phần.

- Backbone MPLS gồm 2 router P (Core) và 3 router PE – đủ để minh họa cơ chế Push/Swap/Pop.

Giới hạn:

- Số liệu đo lường trên Mininet là mô phỏng phần mềm; có thể khác với môi trường sản xuất thực.
- Không triển khai đầy đủ MP-BGP và VPLS do hạn chế của môi trường Mininet cơ bản.
- Mô phỏng tập trung vào MPLS Label Switching và so sánh kiến trúc LAN theo yêu cầu đề tài.

4. Cấu trúc báo cáo

Báo cáo được tổ chức thành sáu chương:

- **Chương 1** (hiện tại): Giới thiệu tổng quan, mục tiêu và phạm vi nghiên cứu.
- **Chương 2:** Cơ sở lý thuyết về kiến trúc LAN, MPLS, Metro Ethernet và các chỉ số hiệu năng.
- **Chương 3:** Thiết kế mô hình mạng – topology logic và bảng địa chỉ IP chi tiết.
- **Chương 4:** Triển khai trên Mininet – môi trường, code topology và hướng dẫn chạy.
- **Chương 5:** Kết quả đo lường, biểu đồ so sánh và phân tích.
- **Chương 6:** Kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

1. Kiến trúc mạng nội bộ (LAN Architectures)

Mạng nội bộ tại mỗi chi nhánh doanh nghiệp có thể được tổ chức theo nhiều mô hình khác nhau, mỗi mô hình có những ưu điểm và nhược điểm riêng tùy vào quy mô và yêu cầu của tổ chức.

1.1. Mạng phẳng (Flat Network)

Mạng phẳng là mô hình đơn giản nhất: tất cả thiết bị đầu cuối (host, máy in, server) được kết nối chung vào một hoặc một vài switch Layer 2, chia sẻ cùng một **broadcast domain** (miền quảng bá).

Đặc điểm:

- Tất cả thiết bị nằm trong cùng một subnet (ví dụ: 10.1.0.0/24).
- Giao tiếp nội bộ hoàn toàn ở Layer 2 (không cần routing).
- Cấu hình đơn giản, chi phí thấp.

Ưu điểm: Triển khai nhanh, quản lý đơn giản, phù hợp mạng quy mô nhỏ (dưới 50 thiết bị).

Nhược điểm:

- **Broadcast storm:** Khi số lượng thiết bị tăng, lưu lượng broadcast (ARP, DHCP...) chiếm dụng băng thông đáng kể.
- Không có phân vùng bảo mật giữa các phòng ban.
- Khả năng mở rộng kém – không phù hợp khi mạng vượt quá vài trăm host.

1.2. Mạng 3 lớp truyền thống (Core – Distribution – Access)

Đây là mô hình phân cấp kinh điển của Cisco, phù hợp cho các tổ chức có quy mô vừa đến lớn.

- **Access Layer (Tầng truy cập):** Kết nối trực tiếp thiết bị đầu cuối. Switch tầng này thực hiện port security, QoS marking và VLAN tagging.
- **Distribution Layer (Tầng phân phối):** Tập hợp các switch access, thực hiện **Inter-VLAN Routing** (định tuyến giữa các VLAN), áp dụng chính sách bảo mật (ACL), và tóm tắt định tuyến lên Core.
- **Core Layer (Tầng lõi):** Chuyển mạch tốc độ cực cao, kết nối các khối Distribution

với nhau và với gateway ra ngoài (CE Router). Không thực hiện chính sách phức tạp để đảm bảo tốc độ.

Ưu điểm: Phân vùng rõ ràng, dễ debug, hỗ trợ VLAN và QoS, khả năng mở rộng tốt, dự phòng linh hoạt (STP, HSRP).

Nhược điểm: Chi phí cao hơn mạng phẳng; STP có thể chặn đường dự phòng, lãng phí băng thông; độ trễ cao hơn do gói tin phải đi qua nhiều lớp switch.

1.3. Mạng 2 lớp Leaf-Spine

Kiến trúc Leaf-Spine là mô hình hiện đại được phát triển cho **Data Center** và các mạng đòi hỏi băng thông cực cao, độ trễ thấp.

- **Spine (Lõi):** Các switch lõi tốc độ cao. Mỗi Spine kết nối với *tất cả* Leaf switch (Full Mesh). Spine không kết nối trực tiếp với thiết bị đầu cuối.
- **Leaf (Truy cập):** Các switch truy cập kết nối trực tiếp host. Mỗi Leaf kết nối với tất cả Spine.

Ưu điểm:

- **Độ trễ nhất quán:** Mọi gói tin giữa hai host đều đi qua tối đa 2 hop (Leaf → Spine → Leaf) – không phụ thuộc vị trí.
- **Băng thông cực lớn:** Full Mesh Spine-Leaf cho phép ECMP (Equal-Cost Multi-Path) chia tải đều trên nhiều đường.
- **Không cần STP:** Kiến trúc không tạo vòng lặp Layer 2, loại bỏ hạn chế của Spanning Tree.
- **Mở rộng dễ dàng:** Thêm Leaf switch không cần thay đổi cấu hình Spine.

Nhược điểm: Chi phí cao hơn (cần nhiều cổng uplink trên mỗi Leaf), phức tạp hơn khi cấu hình định tuyến Layer 3 (ECMP, OSPF/BGP).

2. Công nghệ MPLS (Multiprotocol Label Switching)

2.1. Khái niệm và nguyên lý

MPLS là công nghệ **chuyển mạch nhãn** hoạt động ở *Layer 2.5* (giữa Layer 2 và Layer 3). Thay vì mỗi router phải tra cứu địa chỉ IP đích trong bảng định tuyến (tốn kém về thời gian CPU), MPLS chỉ cần đọc một **nhãn (label)** gắn vào header của gói tin để đưa ra quyết định chuyển tiếp – nhanh hơn nhiều.

Một nhãn MPLS có cấu trúc 32-bit gồm:

- **Label (20 bit):** Giá trị nhãn (0–1.048.575).

- **TC/EXP (3 bit):** Traffic Class – dùng cho QoS.
- **S (1 bit):** Bottom of Stack – nhãn cuối cùng trong ngăn xếp.
- **TTL (8 bit):** Time To Live.

2.2. Cơ chế Push – Swap – Pop

1. **Push (Gắn nhãn):** Khi gói IP từ CE đến PE vào (Ingress PE), router PE tra bảng MPLS forwarding (LFIB) và *gắn một nhãn* vào header gói tin. Gói tin bây giờ là MPLS packet.
2. **Swap (Hoán đổi nhãn):** Các router P ở lõi backbone nhận MPLS packet, *thay nhãn cũ bằng nhãn mới* (theo LFIB của mình) rồi chuyển tiếp. Không cần tra IP header.
3. **Pop (Gỡ nhãn):** Router PE đích (Egress PE) nhận MPLS packet, *gỡ nhãn* và trả lại gói IP thuần túy để chuyển tiếp về CE đích. Kỹ thuật **Penultimate Hop Popping (PHP)** cho phép router P cuối cùng gỡ nhãn giúp PE, giảm tải cho PE.

2.3. So sánh MPLS với IP Routing truyền thống

Bảng 2.1: So sánh MPLS và IP Routing truyền thống

Tiêu chí	IP Routing truyền thống	MPLS Label Switching
Cơ sở quyết định	Tra bảng định tuyến IP (longest prefix match)	Tra bảng nhãn LFIB (exact match – nhanh hơn)
Tốc độ xử lý	Chậm hơn do xử lý IP header	Nhanh hơn do chỉ đọc label 20-bit
Lớp hoạt động	Layer 3	Layer 2.5
Traffic Engineering	Hạn chế (chỉ theo routing metric)	Linh hoạt (RSVP-TE, đặt trước bảng thông)
VPN / Phân tách lưu lượng	Cần VRF + nhiều cấu hình	Tích hợp sẵn (MPLS VPN, VPLS)
Giao thức phân phối nhãn	Không có	LDP, RSVP-TE, MP-BGP
Độ phức tạp cấu hình	Đơn giản hơn	Phức tạp hơn, cần hiểu LFIB/FEC/LSP

3. Metro Ethernet và VPLS

3.1. Metro Ethernet

Metro Ethernet (hay Carrier Ethernet) là công nghệ cung cấp dịch vụ kết nối Ethernet dạng thương mại ở quy mô đô thị (Metropolitan Area). Khách hàng (doanh nghiệp) *cảm thấy* như đang cắm dây vào một Switch khổng lồ duy nhất, trong khi thực tế ISP dùng hạ tầng phức tạp ở bên dưới.

Các loại dịch vụ Metro Ethernet (MEF):

- **E-Line (Point-to-Point):** Kết nối điểm-điểm giữa 2 chi nhánh – tương đương đường leased line Ethernet.
- **E-LAN (Multipoint-to-Multipoint):** Kết nối nhiều chi nhánh – tất cả đặt trong cùng một "LAN ảo" khổng lồ (đây là dịch vụ đề tài này hướng tới).
- **E-Tree:** Kết nối hình cây – root site giao tiếp với nhiều leaf site.

3.2. VPLS (Virtual Private LAN Service)

VPLS là công nghệ **Layer 2 VPN** được xây dựng trên nền MPLS, cho phép nhiều chi nhánh chia sẻ một "LAN ảo" duy nhất xuyên qua mạng ISP. Các frame Ethernet (kể cả broadcast, VLAN tag) được đóng gói trong **Pseudowire** (đường hầm ảo) đi qua lõi MPLS.

Vai trò trong đề tài: Do giới hạn của Mininet, đề tài mô phỏng MPLS Label Switching ở mức IP routing thay vì VPLS Layer 2. Tuy nhiên, ý tưởng kết nối đa chi nhánh như một E-LAN vẫn được thể hiện qua mô hình định tuyến xuyên backbone.

4. Các giao thức điều khiển MPLS

4.1. OSPF (Open Shortest Path First)

OSPF là giao thức định tuyến nội vùng (IGP) dựa trên thuật toán Dijkstra (SPF). Trong mạng MPLS:

- OSPF chạy giữa các router PE và P để tất cả "*nhìn thấy*" nhau (underlay connectivity).
- OSPF cung cấp topology database để LDP biết đường đi và phân phối nhãn theo.
- Hỗ trợ phân vùng theo Area để giảm tải tính toán SPF trong mạng lớn.

4.2. LDP (Label Distribution Protocol)

LDP tự động phân phối nhãn MPLS giữa các router:

- Mỗi router thiết lập **LDP session** TCP với các router láng giềng.
- LDP ánh xạ mỗi **FEC (Forwarding Equivalence Class)** – thường là một prefix IP – với một nhãn cục bộ.
- Nhãn được trao đổi giữa các router để xây dựng **LSP (Label Switched Path)**.

4.3. MP-BGP (Multiprotocol BGP)

MP-BGP mở rộng BGP để mang thêm thông tin VPN (VPN-IPv4 prefix, Route Target). Trong MPLS VPN:

- PE router dùng MP-BGP để trao đổi routing information của khách hàng với các PE khác.
- P router không cần chạy MP-BGP (chỉ cần OSPF + LDP) – đây là ưu điểm giúp P router nhẹ hơn.
- Route Target (RT) và Route Distinguisher (RD) dùng để phân tách bảng định tuyến của từng khách hàng (VRF).

5. Các chỉ số hiệu năng mạng

Bảng 2.2: Định nghĩa và đơn vị đo các chỉ số hiệu năng

Chỉ số	Đơn vị	Định nghĩa	Công cụ đo
Throughput (Thông lượng)	Mbps, Gbps	Lượng dữ liệu thực tế truyền qua mạng trong một đơn vị thời gian. Phản ánh hiệu quả sử dụng băng thông.	iperf3 (TCP mode)
Delay/Latency (Độ trễ)	ms	Thời gian gói tin đi từ nguồn đến đích (RTT/2 cho one-way delay). Bao gồm: propagation delay, transmission delay, queuing delay.	ping (ICMP)
Packet Loss (Mất gói)	%	Tỷ lệ phần trăm gói tin bị mất trong quá trình truyền. Mất gói xảy ra khi bộ đệm router bị đầy (congestion) hoặc lỗi đường truyền.	ping, iperf3 (UDP)
Jitter (Biến động trễ)	ms	Sự biến thiên của delay giữa các gói tin liên tiếp. Jitter cao gây giật lag trong ứng dụng real-time (VoIP, video call).	iperf3 (UDP), mdev trong ping

Mối quan hệ giữa các chỉ số: Jitter được tính toán từ biến thiên delay:

$$J_i = |D_i - D_{i-1}|, \quad \bar{J} = \frac{1}{N-1} \sum_{i=2}^N |D_i - D_{i-1}|$$

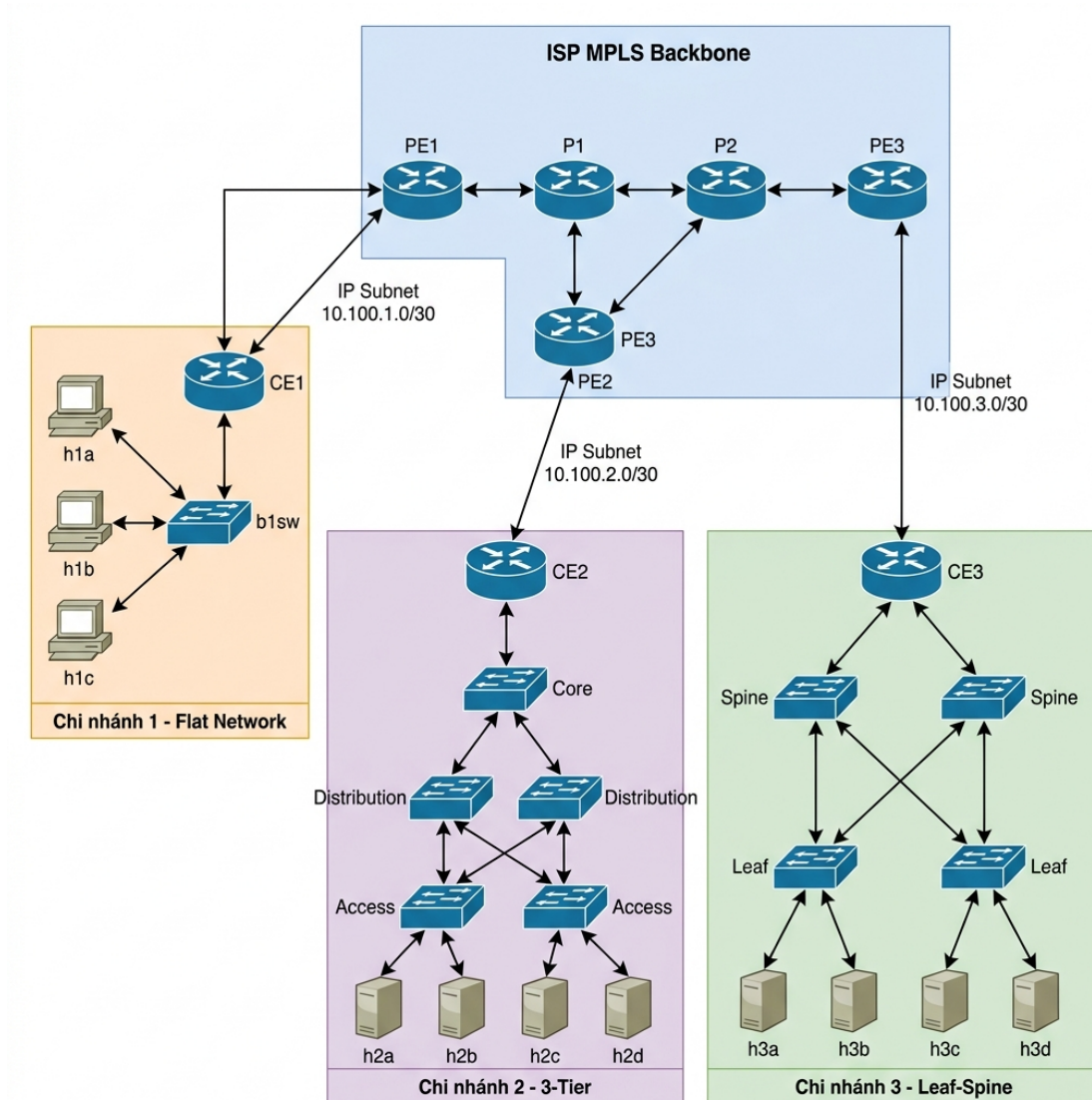
Trong đó D_i là delay của gói tin thứ i , $\bar{J}$ là jitter trung bình.

CHƯƠNG 3. THIẾT KẾ MÔ HÌNH MẠNG

1. Tổng quan thiết kế

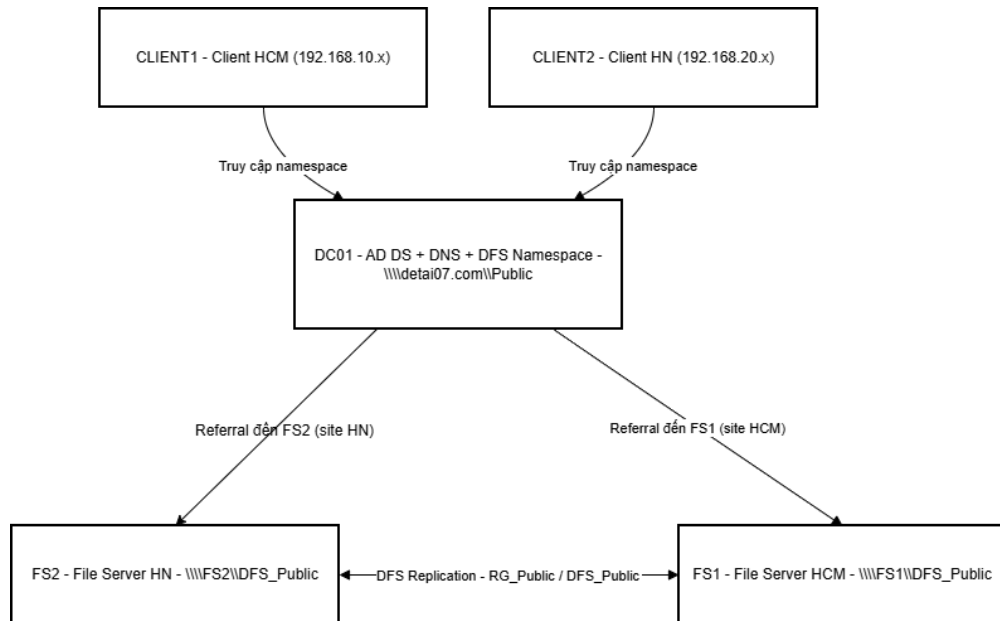
Mô hình mạng được thiết kế gồm hai phần chính:

1. **Mạng nội bộ (LAN) tại 3 chi nhánh:** Mỗi chi nhánh dùng một kiến trúc LAN khác nhau để so sánh hiệu năng.
2. **Backbone Metro Ethernet MPLS của ISP:** Các router CE, PE, P tạo thành hạ tầng kết nối xuyên chi nhánh.



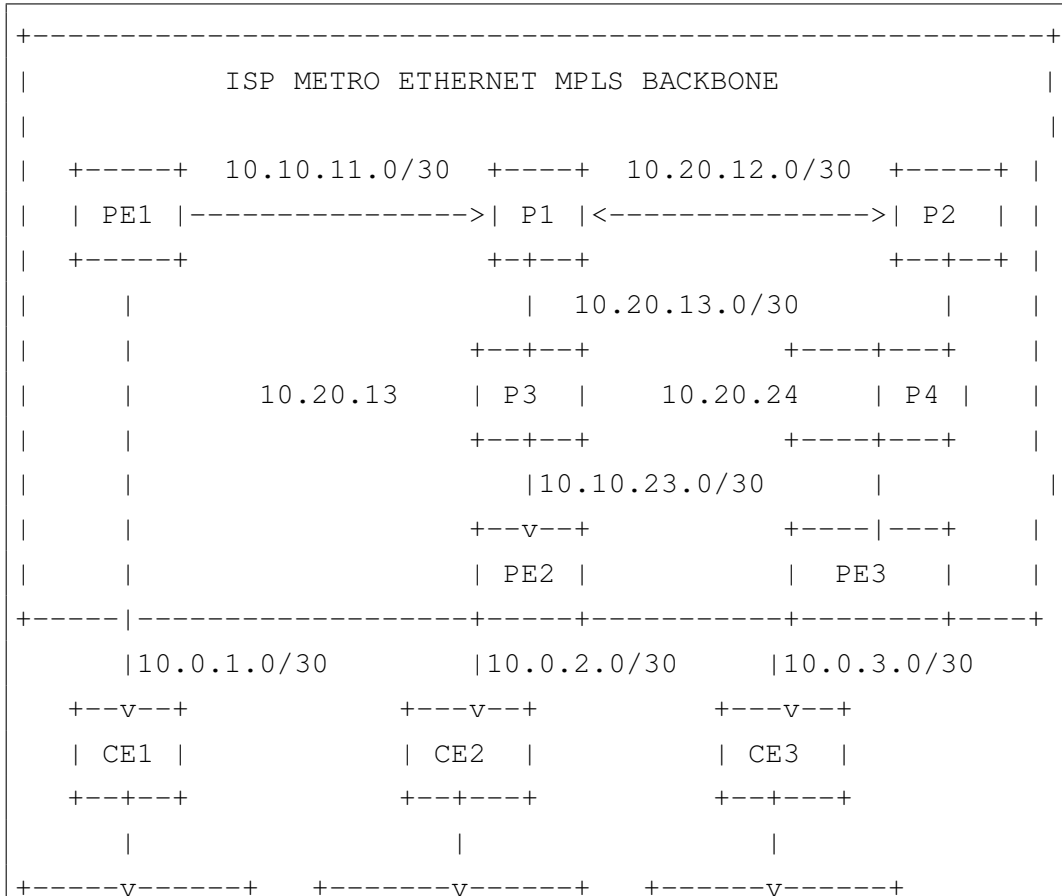
Hình 3.1: Sơ đồ tổng thể mô hình Metro Ethernet MPLS kết nối 3 chi nhánh

1.1. Sơ đồ logic tổng thể



Hình 3.2: Sơ đồ kiến trúc tổng thể hệ thống

Cấu trúc logic tổng thể hệ thống được mô tả như sau:



Branch 1	Branch 2	Branch 3
Flat Net	3-Tier CDA	Leaf-Spine
host1-4	admin,lab,	web,dns,db
10.1.0.0/24	guest hosts	servers
+-----+	+-----+	+-----+

2. Thiết kế chi nhánh 1: Mạng phẳng (Flat Network)

2.1. Cấu trúc topology

Chi nhánh 1 sử dụng kiến trúc mạng phẳng đơn giản nhất:

```

CE1 (gateway: 10.1.0.1/24)
|
[s_acc1 - Access Switch Layer 2]
|-----|-----|
host1      host2      host3      host4
10.1.0.101 10.1.0.102 10.1.0.103 10.1.0.104

```

Mô tả:

- 1 switch Layer 2 duy nhất (s_acc1) kết nối tất cả host.
- 4 host: host1, host2, host3, host4 – cùng subnet 10.1.0.0/24.
- CE1_lan đóng vai trò default gateway và kết nối lên PE1.
- Không có VLAN, không có routing nội bộ.

2.2. Thông số kỹ thuật

- Số switch: 1 (OVS, failMode=standalone)
- Số host: 4 (host1, host2, host3, host4)
- Bảng thông host-switch: không giới hạn (virtual link)
- Subnet: 10.1.0.0/24, Gateway: 10.1.0.1

3. Thiết kế chi nhánh 2: Mạng 3 lớp (Core–Distribution–Access)

3.1. Cấu trúc topology

```

CE2_LAN (Inter-VLAN Router)
|-----|
core1      core2
|-----|   |-----|

```

```

      dist1      dist2
      /---\  \   /  /---\
      acc1  acc2 acc2  acc3
        |      |      |
      admin1 lab1-2  guest1-2
      admin2
VLAN10: admin (10.2.10.0/24)
VLAN20: lab   (10.2.20.0/24)
VLAN30: guest (10.2.30.0/24)

```

Mô tả:

- **Access switches** (acc1, acc2, acc3): Kết nối host, mỗi switch 2 host.
- **Distribution switches** (dist1, dist2): Tập hợp uplink từ access, cross-connected để dự phòng.
- **Core switches** (core1, core2): Kết nối distribution lên CE2_lan (dual uplink).
- CE2_lan: Inter-VLAN routing giữa 3 VLAN (Admin/Lab/Guest) + kết nối lên PE2.

3.2. Thông số kỹ thuật

- Số switch: 7 (Core x2 + Dist x2 + Access x3)
- Số host: 6 (admin1-2, lab1-2, guest1-2)
- VLAN10=10.2.10.0/24, VLAN20=10.2.20.0/24, VLAN30=10.2.30.0/24
- Link Core-CE2_lan: dual uplink (10.2.0.0/30 và 10.2.0.4/30)

4. Thiết kế chi nhánh 3: Mạng Leaf-Spine

4.1. Cấu trúc topology

```

      CE3_LAN (gateway + dual uplink)
      |-----|
[spine1]-----[spine2]    <- Spine (loại, full-mesh)
  /  |  \          /  |  \
lf1  lf2  lf3 lf1 lf2 lf3  <- Leaf (truy cập, mỗi leaf nối cả 2 spine)
  |      |      |
web  dns  db          <- Server zones
1-2  1-2  1-2

Web zone : 10.3.10.0/24
DNS zone : 10.3.20.0/24

```

DB zone : 10.3.30.0/24

Mô tả:

- 2 **Spine** switch (spine1, spine2): Lõi băng thông cao, kết nối full-mesh với mọi Leaf.
- 3 **Leaf** switch (leaf1, leaf2, leaf3): Truy cập server, mọi Leaf kết nối cả 2 Spine.
- Full Mesh Spine-Leaf cho phép ECMP (Equal-Cost Multi-Path).
- CE3_lan kết nối cả hai Spine để tăng dự phòng.

4.2. Thông số kỹ thuật

- Số switch: 5 (Spine x2 + Leaf x3)
- Số host: 6 (web1-2, dns1-2, db1-2)
- Mọi Leaf có 2 uplink lên Spine (redundant path)
- CE3_lan: dual uplink lên spine1 (10.3.0.0/30) và spine2 (10.3.0.4/30)

5. Thiết kế backbone MPLS của ISP**5.1. Topology backbone**

Backbone MPLS (4P + 3PE):

```

PE1 ---[10.10.11.0/30]--- P1 ---[10.20.12.0/30]--- P2 ---[10.10.32.0/30]---
      |                               |                               |
      |   [10.20.13]   |   [10.20.23]   |
      |               P3 --[10.10.23.0/30]-- PE2
      |               |
      |   [10.20.14/10.20.24/10.20.34]
      |               P4
      +---[10.10.13.0/30]---+---[10.10.24.0/30]--PE2
                              +---[10.10.34.0/30]--PE3
  
```

Mô tả thiết bị:

- **PE1**: Kết nối CE1 (chi nhánh 1). Push label khi gói vào, nhận label khi gói ra.
- **PE2**: Kết nối CE2 (chi nhánh 2). Dual uplink lên P3 và P4.
- **PE3**: Kết nối CE3 (chi nhánh 3). Dual uplink lên P2 và P4.
- **P1, P2, P3, P4**: Provider Core – chỉ Swap label, kết nối full-mesh với nhau.

5.2. Kế hoạch nhãn MPLS (Label Plan)

Bảng 3.1: Bảng phân phối nhãn MPLS (LFIB – Label Forwarding Information Base)

Node	Thao tác	Mục tiêu	Label vào	Label ra
PE1	PUSH	10.2.0.0/16 (LAN2) via P1	–	200
PE1	PUSH	10.3.0.0/16 (LAN3) via P1	–	300
PE2	PUSH	10.1.0.0/24 (LAN1) via P3	–	100
PE2	PUSH	10.3.0.0/16 (LAN3) via P4	–	310
PE3	PUSH	10.1.0.0/24 (LAN1) via P2	–	110
PE3	PUSH	10.2.0.0/16 (LAN2) via P2	–	210
P1	SWAP	forward sang P3 (LAN1→LAN2)	200	201
P1	SWAP	forward sang P2 (LAN1→LAN3)	300	301
P2	SWAP	forward sang P1 (LAN3→LAN1)	110	111
P2	SWAP	forward sang P4 (LAN3→LAN2)	210	211
P3	SWAP	forward sang P1 (LAN2→LAN1)	100	101
P3	SWAP	forward sang P4 (LAN2→LAN3)	310	311
P1	POP	giao về PE1 (label=0)	101,111	0
P3	POP	giao về PE2 (label=0)	201	0
P2	POP	giao về PE3 (label=0)	301	0
P4	POP	giao về PE2/PE3	211,311	0

6. Bảng địa chỉ IP toàn hệ thống

Bảng 3.2: Bảng địa chỉ IP – LAN chi nhánh

Chi nhánh	Host	địa chỉ IP	Subnet	Default GW
1 – Flat	host1	10.1.0.101	/24	10.1.0.1
	host2	10.1.0.102	/24	10.1.0.1
	host3	10.1.0.103	/24	10.1.0.1
	host4	10.1.0.104	/24	10.1.0.1
	CE1_lan	10.1.0.1	/24	–
2 – 3-Tier	admin1	10.2.10.11	/24	10.2.10.1
	admin2	10.2.10.12	/24	10.2.10.1
	lab1	10.2.20.21	/24	10.2.20.1
	lab2	10.2.20.22	/24	10.2.20.1
	guest1	10.2.30.31	/24	10.2.30.1
	guest2	10.2.30.32	/24	10.2.30.1
	CE2_lan-gw	10.2.x.1	/24	–
3 – Leaf-Spine	web1	10.3.10.11	/24	10.3.10.1
	web2	10.3.10.12	/24	10.3.10.1
	dns1	10.3.20.21	/24	10.3.20.1
	dns2	10.3.20.22	/24	10.3.20.1
	db1	10.3.30.31	/24	10.3.30.1
	db2	10.3.30.32	/24	10.3.30.1
	CE3_lan-gw	10.3.x.1	/24	–

Bảng 3.3: Bảng địa chỉ IP – Backbone ISP MPLS

Segment	Router A	Router B	Subnet	
CE1 – PE1	ce1-eth0: 10.0.1.1	pe1-eth0: 10.0.1.2	/30	
CE2 – PE2	ce2-eth0: 10.0.2.1	pe2-eth0: 10.0.2.2	/30	
CE3 – PE3	ce3-eth0: 10.0.3.1	pe3-eth0: 10.0.3.2	/30	
CE1 – CE1_lan	ce1-eth1: 10.100.1.1	ce1_lan-eth1: 10.100.1.2	/30	
CE2 – CE2_lan	ce2-eth1: 10.100.2.1	ce2_lan-eth2: 10.100.2.2	/30	
CE3 – CE3_lan	ce3-eth1: 10.100.3.1	ce3_lan-eth2: 10.100.3.2	/30	
PE1 – P1	pe1-eth1: 10.10.11.1	p1-eth0: 10.10.11.2	/30	
PE1 – P3	pe1-eth2: 10.10.13.1	p3-eth0: 10.10.13.2	/30	
PE2 – P3	pe2-eth1: 10.10.23.1	p3-eth1: 10.10.23.2	/30	
PE2 – P4	pe2-eth2: 10.10.24.1	p4-eth0: 10.10.24.2	/30	
PE3 – P2	pe3-eth1: 10.10.32.1	p2-eth0: 10.10.32.2	/30	
PE3 – P4	pe3-eth2: 10.10.34.1	p4-eth1: 10.10.34.2	/30	
P1 – P2	p1-eth1: 10.20.12.1	p2-eth1: 10.20.12.2	/30	
P1 – P3	p1-eth2: 10.20.13.1	p3-eth2: 10.20.13.2	/30	
P1 – P4	p1-eth3: 10.20.14.1	p4-eth2: 10.20.14.2	/30	
P2 – P3	p2-eth2: 10.20.23.1	p3-eth3: 10.20.23.2	/30	
P2 – P4	p2-eth3: 10.20.24.1	p4-eth3: 10.20.24.2	/30	
P3 – P4	p3-eth4: 10.20.34.1	p4-eth4: 10.20.34.2	/30	

CHƯƠNG 4. TRIỂN KHAI TRÊN MININET

1. Môi trường cài đặt

1.1. Yêu cầu hệ thống

Bảng 4.1: Yêu cầu phần mềm và phần cứng

Thành phần	Phiên bản / Yêu cầu
Hệ điều hành	Ubuntu 22.04 LTS (64-bit)
RAM	Tối thiểu 4 GB (khuyến nghị 8 GB)
CPU	2 cores trở lên
Disk	20 GB trống
Python	Python 3.10+
Mininet	2.3.0+
Open vSwitch	2.17+
iperf3	3.9+
iproute2	Hỗ trợ MPLS (kernel 4.1+)

1.2. Hướng dẫn cài đặt từng bước

Bước 1: Cập nhật hệ thống

```
sudo apt update && sudo apt upgrade -y
```

Listing 4.1: Cập nhật hệ thống Ubuntu

Bước 2: Cài đặt các gói cần thiết

```
sudo apt install -y \
  mininet \
  openvswitch-switch openvswitch-common \
  python3 python3-pip \
  iperf3 iperf \
  tcpdump net-tools iproute2 \
  iputils-ping traceroute \
  python3-matplotlib python3-numpy \
  curl wget git
```

Listing 4.2: Cài đặt Mininet và các công cụ hỗ trợ

Bước 3: Kích hoạt MPLS kernel module

```
# Load module MPLS
```



```

sudo modprobe mpls_router
sudo modprobe mpls_ip tunnel
sudo modprobe mpls_gso

# Kiểm tra module đã load:
lsmod | grep mpls

# Bật platform labels (cần thiết cho iproute2 MPLS)
echo 'net.mpls.platform_labels = 100000' | \
    sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# Xác nhận:
sysctl net.mpls.platform_labels

```

Listing 4.3: Load module MPLS và bật platform labels

Bước 4: Kiểm tra Mininet hoạt động

```

# Test đơn giản: topo single, 3 với ping
sudo mn --topo single,3 --test ping

# Kết quả mong đợi:
# *** Ping: testing ping reachability
# h1 -> h2 h3
# h2 -> h1 h3
# h3 -> h1 h2
# *** Results: 0% dropped (6/6 received)

# Đơn giản sau test
sudo mn -c

```

Listing 4.4: Kiểm tra Mininet với topo đơn giản

2. Cấu trúc file code

Cây thờ mục dự án

```

TKMCK/
+-- topology/
|   +-- metro_full.py      <- Orchestrator tổng thể (file chính)
|   +-- configure_mpls.py  <- Cấu hình MPLS Push/Swap/Pop
|   +-- topo_branch1_flat.py
|   +-- topo_branch2_3tier.py
|   +-- topo_branch3_leafspine.py
|   +-- topo_backbone_mpls.py
+-- scripts/

```

```

|   +-- run_measurements.py <- Do luong tu dong
|   +-- plot_results.py      <- Ve bieu do
|   +-- parse_ping.py
|   +-- parse_iperf.py
|   +-- aggregate_results.py
+-- config/
|   +-- tests.yaml           <- Cau hinh kich ban do luong
+-- results/
|   +-- raw/                 <- Log thu
|   +-- csv/                 <- Du lieu tong hop
|   +-- charts/              <- Bieu do PNG
+-- source_latex/            <- Bao cao LaTeX
+-- README.md

```

3. Mã nguồn chi tiết

3.1. Lớp LinuxRouter và MetroFullTopo

Lớp LinuxRouter kế thừa từ Node của Mininet, bật ip_forward để node hoạt động nhờ router Linux thật sự:

```

1 class LinuxRouter(Node):
2     """Node bat ip_forward -> hoat dong nhu Linux router."""
3     def config(self, **params):
4         super().config(**params)
5         self.cmd('sysctl -w net.ipv4.ip_forward=1')
6
7     def terminate(self):
8         self.cmd('sysctl -w net.ipv4.ip_forward=0')
9         super().terminate()

```

Listing 4.5: Lớp LinuxRouter – node hoạt động nhờ IP router

Lớp MetroFullTopo xây dựng toàn bộ topology theo phống pháp modular, gọi 4 hàm riêng:

```

1 class MetroFullTopo(Topo):
2     def build(self, **opts):
3         self._build_backbone()    # CE/PE/P + P2P links
4         self._build_branch1()     # Flat Network
5         self._build_branch2()     # 3-Tier CDA
6         self._build_branch3()     # Leaf-Spine
7         self._build_glue_links()  # CE <-> CE_LAN
8

```

```

9  def _build_backbone(self):
10     # CE routers (Customer Edge)
11     self.addNode('ce1', cls=LinuxRouter, ip=None)
12     self.addNode('ce2', cls=LinuxRouter, ip=None)
13     self.addNode('ce3', cls=LinuxRouter, ip=None)
14     # PE routers (Provider Edge)
15     self.addNode('pe1', cls=LinuxRouter, ip=None)
16     self.addNode('pe2', cls=LinuxRouter, ip=None)
17     self.addNode('pe3', cls=LinuxRouter, ip=None)
18     # P routers (Provider Core - MPLS region)
19     self.addNode('p1', cls=LinuxRouter, ip=None)
20     self.addNode('p2', cls=LinuxRouter, ip=None)
21     self.addNode('p3', cls=LinuxRouter, ip=None)
22     self.addNode('p4', cls=LinuxRouter, ip=None)
23     # CE-PE links (point-to-point /30)
24     self.addLink('ce1', 'pe1',
25                 intfName1='ce1-eth0', params1={'ip': '10.0.1.1/30'},
26                 intfName2='pe1-eth0', params2={'ip': '10.0.1.2/30'})
27     # ... (tuong tu cho ce2-pe2, ce3-pe3)
28     # PE-P links
29     self.addLink('pe1', 'p1',
30                 intfName1='pe1-eth1', params1={'ip': '10.10.11.1/30'},
31                 intfName2='p1-eth0', params2={'ip': '10.10.11.2/30'})
32     # P-P links (full-mesh P1/P2/P3/P4)
33     self.addLink('p1', 'p2',
34                 intfName1='p1-eth1', params1={'ip': '10.20.12.1/30'},
35                 intfName2='p2-eth1', params2={'ip': '10.20.12.2/30'})
36     # ... (tuong tu cho P1-P3, P1-P4, P2-P3, P2-P4, P3-P4)

```

Listing 4.6: Lớp MetroFullTopo – xây dựng topology modular

3.2. Xây dựng chi nhánh 1 – Flat Network

```

1  def _build_branch1(self):
2      """Switch access + 4 host + ce1_lan. Subnet: 10.1.0.0/24."""
3      self.addNode('ce1_lan', cls=LinuxRouter, ip=None)
4      s_acc1 = self.addSwitch('s_acc1',
5                              cls=OVSSwitch,
6                              failMode='standalone')
7      # 4 hosts voi default route qua ce1_lan
8      self.addHost('host1', ip='10.1.0.101/24',
9                  defaultRoute='via 10.1.0.1')
10     self.addHost('host2', ip='10.1.0.102/24',
11                 defaultRoute='via 10.1.0.1')
12     self.addHost('host3', ip='10.1.0.103/24',

```

```

13         defaultRoute='via 10.1.0.1')
14     self.addHost('host4', ip='10.1.0.104/24',
15                 defaultRoute='via 10.1.0.1')
16     # host <-> switch access
17     for h in ['host1', 'host2', 'host3', 'host4']:
18         self.addLink(h, s_acc1)
19     # switch access <-> ce1_lan (gateway LAN)
20     self.addLink(s_acc1, 'ce1_lan',
21                 intfName2='ce1_lan-eth0',
22                 params2={'ip': '10.1.0.1/24'})

```

Listing 4.7: Hàm _build_branch1() – Flat Network

3.3. Xây dựng chi nhánh 2 – 3-Tier CDA

```

1 def _build_branch2(self):
2     """3 lop: core1/core2 -> dist1/dist2 -> acc1/acc2/acc3 -> hosts.
3     VLAN10=admin, VLAN20=lab, VLAN30=guest."""
4     self.addNode('ce2_lan', cls=LinuxRouter, ip=None)
5     # Lop Core (2 switch de du phong)
6     core1 = self.addSwitch('core1', cls=OVSSwitch, failMode='standalone')
7     core2 = self.addSwitch('core2', cls=OVSSwitch, failMode='standalone')
8     # Lop Distribution
9     dist1 = self.addSwitch('dist1', cls=OVSSwitch, failMode='standalone')
10    dist2 = self.addSwitch('dist2', cls=OVSSwitch, failMode='standalone')
11    # Lop Access
12    acc1 = self.addSwitch('acc1', cls=OVSSwitch, failMode='standalone')
13    acc2 = self.addSwitch('acc2', cls=OVSSwitch, failMode='standalone')
14    acc3 = self.addSwitch('acc3', cls=OVSSwitch, failMode='standalone')
15    # Hosts (3 VLAN khac nhau)
16    self.addHost('admin1', ip='10.2.10.11/24',
17                defaultRoute='via 10.2.10.1')
18    self.addHost('admin2', ip='10.2.10.12/24',
19                defaultRoute='via 10.2.10.1')
20    self.addHost('lab1', ip='10.2.20.21/24',
21                defaultRoute='via 10.2.20.1')
22    self.addHost('guest1', ip='10.2.30.31/24',
23                defaultRoute='via 10.2.30.1')
24    # ce2_lan <-> Core (dual uplink)
25    self.addLink('ce2_lan', core1,
26                intfName1='ce2_lan-eth0', params1={'ip': '10.2.0.1/30'})
27    self.addLink('ce2_lan', core2,
28                intfName1='ce2_lan-eth1', params1={'ip': '10.2.0.5/30'})
29    # Core <-> Distribution (cross-connected)
30    self.addLink(core1, dist1); self.addLink(core1, dist2)

```

```

31 self.addLink(core2, dist1); self.addLink(core2, dist2)
32 # Distribution <-> Access
33 self.addLink(dist1, acc1); self.addLink(dist1, acc2)
34 self.addLink(dist2, acc2); self.addLink(dist2, acc3)

```

Listing 4.8: Hàm _build_branch2() – 3-Tier Core-Distribution-Access

3.4. Xây dựng chi nhánh 3 – Leaf-Spine

```

1 def _build_branch3(self):
2     """2 Spine + 3 Leaf (full-mesh) + 6 servers."""
3     self.addNode('ce3_lan', cls=LinuxRouter, ip=None)
4     spine1 = self.addSwitch('spine1', cls=OVSSwitch, failMode='standalone')
5     spine2 = self.addSwitch('spine2', cls=OVSSwitch, failMode='standalone')
6     leaf1 = self.addSwitch('leaf1', cls=OVSSwitch, failMode='standalone')
7     leaf2 = self.addSwitch('leaf2', cls=OVSSwitch, failMode='standalone')
8     leaf3 = self.addSwitch('leaf3', cls=OVSSwitch, failMode='standalone')
9     # Server zones
10    self.addHost('web1', ip='10.3.10.11/24',
11                defaultRoute='via 10.3.10.1')
12    self.addHost('dns1', ip='10.3.20.21/24',
13                defaultRoute='via 10.3.20.1')
14    self.addHost('db1', ip='10.3.30.31/24',
15                defaultRoute='via 10.3.30.1')
16    # ce3_lan <-> Spine (dual uplink)
17    self.addLink('ce3_lan', spine1,
18                intfName1='ce3_lan-eth0', params1={'ip': '10.3.0.1/30'})
19    self.addLink('ce3_lan', spine2,
20                intfName1='ce3_lan-eth1', params1={'ip': '10.3.0.5/30'})
21    # Spine <-> Leaf (full-mesh: moi leaf noi TAT CA spine)
22    for spine in [spine1, spine2]:
23        for leaf in [leaf1, leaf2, leaf3]:
24            self.addLink(spine, leaf)
25    # Leaf <-> Servers
26    self.addLink(leaf1, 'web1'); self.addLink(leaf1, 'web2')
27    self.addLink(leaf2, 'dns1'); self.addLink(leaf2, 'dns2')
28    self.addLink(leaf3, 'db1'); self.addLink(leaf3, 'db2')

```

Listing 4.9: Hàm _build_branch3() – Leaf-Spine (2-tier)

4. Cấu hình MPLS (Push – Swap – Pop)

4.1. Bước 1: Bật MPLS trên các node backbone

```

1 def configure_mpls(net):
2     """Cau hinh Linux Kernel MPLS tren toan backbone."""
3     MPLS_NODES = ['pe1', 'pe2', 'pe3', 'p1', 'p2', 'p3', 'p4']
4     MPLS_INTERFACES = {
5         'pe1': ['pe1-eth1', 'pe1-eth2'],
6         'pe2': ['pe2-eth1', 'pe2-eth2'],
7         'pe3': ['pe3-eth1', 'pe3-eth2'],
8         'p1': ['p1-eth0', 'p1-eth1', 'p1-eth2', 'p1-eth3'],
9         'p2': ['p2-eth0', 'p2-eth1', 'p2-eth2', 'p2-eth3'],
10        'p3': ['p3-eth0', 'p3-eth1', 'p3-eth2', 'p3-eth3', 'p3-eth4'],
11        'p4': ['p4-eth0', 'p4-eth1', 'p4-eth2', 'p4-eth3', 'p4-eth4'],
12    }
13    for name in MPLS_NODES:
14        node = net[name]
15        # Load kernel modules
16        node.cmd('modprobe mpls_router 2>/dev/null || true')
17        node.cmd('modprobe mpls_iptunnel 2>/dev/null || true')
18        # Bat platform labels
19        node.cmd('sysctl -w net.mpls.platform_labels=100000')
20        # Bat MPLS input tren tung interface
21        for iface in MPLS_INTERFACES.get(name, []):
22            node.cmd(f'sysctl -w net.mpls.conf.{iface}.input=1')

```

Listing 4.10: Hàm khởi tạo MPLS trên từng node

4.2. Bước 2: Cấu hình PUSH tại PE routers (Ingress LER)

Khi gói IP từ CE đến PE, router PE sử dụng lệnh `ip route add encap mpls` để gán nhãn MPLS trước khi chuyển tiếp vào mạng backbone:

```

1 def _configure_pe_push(net):
2     """PUSH: PE gan nhan MPLS vao goi IP den tu CE."""
3     pe1, pe2, pe3 = net['pe1'], net['pe2'], net['pe3']
4
5     # PE1: push label cho traffic den LAN2 (label=200) va LAN3 (label
6     # =300)
7     pe1.cmd('ip route add 10.2.0.0/16 encap mpls 200 '
8             'via 10.10.11.2 dev pe1-eth1')
9     pe1.cmd('ip route add 10.3.0.0/16 encap mpls 300 '
10            'via 10.10.11.2 dev pe1-eth1')
11
12    # PE2: push label cho traffic den LAN1 (label=100) va LAN3 (label

```

```

=310)
12 pe2.cmd('ip route add 10.1.0.0/24 encap mpls 100 '
13         'via 10.10.23.2 dev pe2-eth1')
14 pe2.cmd('ip route add 10.3.0.0/16 encap mpls 310 '
15         'via 10.10.24.2 dev pe2-eth2')
16
17 # PE3: push label cho traffic den LAN1 (label=110) va LAN2 (label
=210)
18 pe3.cmd('ip route add 10.1.0.0/24 encap mpls 110 '
19         'via 10.10.32.2 dev pe3-eth1')
20 pe3.cmd('ip route add 10.2.0.0/16 encap mpls 210 '
21         'via 10.10.32.2 dev pe3-eth1')

```

Listing 4.11: Cấu hình PUSH tại PE1, PE2, PE3

4.3. Bối 3: Cấu hình SWAP tại P routers (Transit LSR)

Các router P (lỗi) chỉ đọc nhãn MPLS và hoán đổi sang nhãn mới, không cần đọc IP header:

```

1 def _configure_p_lfib(net):
2     """SWAP: P nhan goi co nhan, thay bang nhan moi, day ra interface
khac."""
3     p1, p2, p3, p4 = net['p1'], net['p2'], net['p3'], net['p4']
4
5     # ---- P1: SWAP ----
6     # Nhan label 200 (tu PE1, huong LAN2): swap->201, forward qua P3
7     p1.cmd('ip -f mpls route add 200 as 201 via inet 10.20.13.2 dev p1-
eth2')
8     # Nhan label 300 (tu PE1, huong LAN3): swap->301, forward qua P2
9     p1.cmd('ip -f mpls route add 300 as 301 via inet 10.20.12.2 dev p1-
eth1')
10    # Label 101 (tu P3): pop va giao ve PE1
11    p1.cmd('ip -f mpls route add 101 as 0 via inet 10.10.11.1 dev p1-
eth0')
12    # Label 111 (tu P2, LAN3->LAN1): pop va giao ve PE1
13    p1.cmd('ip -f mpls route add 111 as 0 via inet 10.10.11.1 dev p1-
eth0')
14
15    # ---- P2: SWAP ----
16    p2.cmd('ip -f mpls route add 110 as 111 via inet 10.20.12.1 dev p2-
eth1')
17    p2.cmd('ip -f mpls route add 210 as 211 via inet 10.20.24.2 dev p2-
eth3')
18    p2.cmd('ip -f mpls route add 301 as 0 via inet 10.10.32.1 dev p2-
eth0')

```

```

19
20 # ---- P3: SWAP ----
21 p3.cmd('ip -f mpls route add 100 as 101 via inet 10.20.13.1 dev p3-
eth2')
22 p3.cmd('ip -f mpls route add 201 as 0 via inet 10.10.23.1 dev p3-
eth1')
23 p3.cmd('ip -f mpls route add 310 as 311 via inet 10.20.34.2 dev p3-
eth4')
24
25 # ---- P4: SWAP ----
26 p4.cmd('ip -f mpls route add 311 as 0 via inet 10.10.34.1 dev p4-
eth1')
27 p4.cmd('ip -f mpls route add 211 as 0 via inet 10.10.24.1 dev p4-
eth0')

```

Listing 4.12: Cấu hình SWAP LFIB tại P1, P2, P3, P4

4.4. Xem bảng MPLS route trên router

Sau khi cấu hình MPLS xong, có thể kiểm tra bảng LFIB trên từng node:

```

# Xem MPLS forwarding table của P1
mininet> p1 ip -f mpls route show

# Ket qua mau:
# 101 as 0 via 10.10.11.1 dev p1-eth0
# 111 as 0 via 10.10.11.1 dev p1-eth0
# 200 as 201 via 10.20.13.2 dev p1-eth2
# 300 as 301 via 10.20.12.2 dev p1-eth1

# Xem IP route voi encap MPLS tai PE1
mininet> pe1 ip route show

# Bat goi tin tren interface pe1-eth1 de xem MPLS header
mininet> pe1 tcpdump -i pe1-eth1 -n mpls

```

Listing 4.13: Kiểm tra bảng MPLS route tại P1

5. HƯỚNG DẪN CHẠY TOPOLOGY

5.1. Khôi dựng topology tổng thể

```

# Vao thu muc topology
cd ~/TKMCK/topology

# Don dep Mininet cu (neu can)

```



```

sudo mn -c

# Chạy topology trong the (3 chi nhánh + MPLS backbone)
sudo python3 metro_full.py

```

Listing 4.14: Chạy toàn bộ hệ thống Metro Ethernet MPLS

5.2. Kiểm tra kết nối trong Mininet CLI

Sau khi topology khởi động, bạn sẽ thấy dấu nhắc mininet>:

```

# Kiểm tra tất cả node
mininet> nodes

# Kiểm tra toàn bộ liên kết
mininet> net

# Ping từ chi nhánh 1 sang chi nhánh 2 (cross-branch)
mininet> host1 ping -c 5 10.2.10.11

# Ping từ chi nhánh 1 sang chi nhánh 3
mininet> host1 ping -c 5 10.3.10.11

# Ping tất cả (cross-branch)
mininet> host1 ping -c 3 10.2.20.21    # -> lab1
mininet> host1 ping -c 3 10.3.20.21    # -> dns1

# Xem bảng định tuyến của CE1
mininet> cel ip route

# Xem bảng MPLS của P1
mininet> p1 ip -f mpls route

# Traceroute từ host1 -> admin1 (xem đường qua backbone)
mininet> host1 traceroute -n 10.2.10.11

```

Listing 4.15: Các lệnh kiểm tra cơ bản trong Mininet CLI

5.3. Kết quả ping end-to-end thực tế

Sau khi hệ thống khởi động và MPLS được cấu hình, kết quả kiểm tra ping cross-branch:

```

*** KIỂM TRA PING END-TO-END (cross-branch)
OK host1->host2    [cung branch1]          loss=0%
OK host1->admin1    [branch1->branch2]       loss=0%

```

OK host1->web1	[branch1->branch3]	loss=0%
OK admin1->dns1	[branch2->branch3]	loss=0%
OK web1->guest1	[branch3->branch2]	loss=0%
OK db1->host1	[branch3->branch1]	loss=0%

Listing 4.16: Kết quả kiểm tra ping end-to-end cross-branch

5.4. đo lường iperf3 trong CLI

```
# Do TCP throughput: host1 -> admin1 (cross-branch)
mininet> admin1 iperf3 -s &
mininet> host1 iperf3 -c 10.2.10.11 -t 10

# Do UDP jitter va packet loss: host1 -> web1
mininet> web1 iperf3 -s &
mininet> host1 iperf3 -c 10.3.10.11 -u -b 100M -t 10

# Thoat va don dep
mininet> exit
sudo mn -c
```

Listing 4.17: đo thông lượng TCP và UDP bằng iperf3

6. Bộ công cụ đo lường tự động

6.1. Tổng quan bộ đo lường

để thu thập số liệu đồng bộ, dự án tích hợp bộ công cụ đo lường tự động bao gồm:

```
1 # Mau cau hinh tests.yaml
2 scenarios:
3   - name: flat
4     arch_name: "Flat Network (Chi nhanh 1)"
5     tests:
6       - src: host1
7         dst_ip: 10.1.0.102
8         description: "host1->host2 [intra-flat]"
9         type: intra
10      - src: host1
11        dst_ip: 10.2.10.11
12        description: "host1->admin1 [flat->3tier]"
13        type: cross
14
15 measurement:
16   ping_count: 20          # So goi ICMP
17   iperf_duration: 10      # Thoi gian do (giay)
```

```

18 bandwidths: [10M, 50M, 100M] # Cac muc tai UDP
19 repeats: 3 # So lan lap lai moi kich ban

```

Listing 4.18: Câu hình kích bản đô lũng trong tests.yaml

```

# Chay do luong va luu ket qua
cd ~/TKMCK
sudo python3 scripts/run_measurements.py \
    --topology topology/metro_full.py \
    --config config/tests.yaml \
    --output results/

# Ket qua luu tai:
# results/csv/results_<timestamp>.csv
# results/raw/all_<timestamp>/
# results/csv/measurements_<timestamp>.log

# Ve bieu do tu CSV
python3 scripts/plot_results.py \
    --csv results/csv/results_20260418_010018.csv
# Bieu do luu tai: results/charts/

```

Listing 4.19: Chạy bộ đô lũng tự động

CHƯƠNG 5. KẾT QUẢ ĐO LƯỜNG VÀ PHÂN TÍCH

1. Phướng pháp đo lường

Các thí nghiệm được thực hiện trên cùng một topology MPLS backbone. Mỗi kích bản được lặp lại **3 lần** và lấy giá trị trung bình.

- **Throughput UDP:** đo bằng `iperf3` chế độ UDP, 3 mức tải: 10 Mbps, 50 Mbps, 100 Mbps, thời gian 10s.
- **Throughput TCP:** đo bằng `iperf3` chế độ TCP, 10 giây.
- **Delay (RTT):** đo bằng lệnh `ping`, 20 gói ICMP, ghi lại min/avg/max/mdev.
- **Packet Loss:** Ghi nhận từ output `ping` (%) và `iperf3` UDP.
- **Jitter:** Tính từ mdev (mean deviation) trong `ping` hoặc `jitter_ms` `iperf3` UDP.

Phân loại kích bản:

- **Intra-branch:** Hai host cùng chi nhánh – đo hiệu năng LAN nội bộ.
- **Cross-branch:** Host thuộc hai chi nhánh khác nhau – đo hiệu năng xuyên backbone MPLS.

2. Kết quả đo lường nội bộ (Intra-branch)

2.1. Chi nhánh 1 – Flat Network

Kết quả ping thực tế (host1 → host2 và host1 → host3):

```
>>> Test: host1->host2 [intra-flat]
Run 1/3: ping...   loss=0.0%   avg=0.066ms
Run 2/3: ping...   loss=0.0%   avg=0.046ms
Run 3/3: ping...   loss=0.0%   avg=0.045ms
Run 1/3: iperf3 UDP 10Mbps...   tput=9.998Mbps   jitter=0.755ms   loss
=0.0%
Run 1/3: iperf3 UDP 50Mbps...   tput=49.996Mbps   jitter=0.174ms   loss
=0.0%
Run 1/3: iperf3 UDP 100Mbps...   tput=99.988Mbps   jitter=0.033ms   loss
=0.0%
Run 1/3: iperf3 TCP...           48256.355 Mbps

[OK] host1->host2 @ 10Mbps:   tput=10.00Mbps   delay=0.0523ms   jitter
=0.7588ms   loss=0.0%
[OK] host1->host2 @ 50Mbps:   tput=50.00Mbps   delay=0.0523ms   jitter
=0.1565ms   loss=0.0%
```

```
[OK] host1->host2 @ 100Mbps: tput=99.98Mbps  delay=0.0523ms  jitter
    =0.0686ms  loss=0.0%

>>> Test: host1->host3 [intra-flat]
    Run 1/3: ping...  loss=0.0%  avg=0.065ms
    Run 2/3: ping...  loss=0.0%  avg=0.046ms
    Run 3/3: ping...  loss=0.0%  avg=0.047ms

[OK] host1->host3 @ 100Mbps: tput=99.97Mbps  delay=0.0527ms  jitter
    =0.1868ms  loss=0.0%
```

Listing 5.1: Log đo lường intra-branch – Flat Network

2.2. Chi nhánh 2 – 3-Tier (Core–Distribution–Access)

```
>>> Test: admin1->admin2 [same-VLAN10]
    Run 1/3: ping...  loss=0.0%  avg=0.060ms
    Run 2/3: ping...  loss=0.0%  avg=0.050ms
    Run 3/3: ping...  loss=0.0%  avg=0.047ms
    Run 1/3: iperf3 TCP... 48340.288 Mbps

[OK] admin1->admin2 [same-VLAN10] @ 100Mbps:
    tput=99.98Mbps  delay=0.0523ms  jitter=0.1074ms  loss=0.0%

>>> Test: admin1->lab1 [inter-VLAN: VLAN10->VLAN20]
    Run 1/3: ping...  loss=0.0%  avg=0.081ms
    Run 2/3: ping...  loss=0.0%  avg=0.075ms
    Run 3/3: ping...  loss=0.0%  avg=0.071ms
    Run 1/3: iperf3 TCP... 31354.341 Mbps

[OK] admin1->lab1 [inter-VLAN] @ 100Mbps:
    tput=99.99Mbps  delay=0.0757ms  jitter=0.0331ms  loss=0.0%

>>> Test: admin1->guest1 [inter-VLAN: VLAN10->VLAN30]
    Run 1/3: ping...  loss=0.0%  avg=0.101ms
    Run 3/3: ping...  loss=0.0%  avg=0.078ms

[OK] admin1->guest1 [inter-VLAN] @ 100Mbps:
    tput=99.99Mbps  delay=0.0850ms  jitter=0.0477ms  loss=0.0067%
```

Listing 5.2: Log đo lường intra-branch – 3-Tier Network

2.3. Chi nhánh 3 – Leaf-Spine

```
>>> Test: web1->web2 [same-leaf1]
    Run 1/3: ping...  loss=0.0%  avg=0.067ms
```

```

Run 2/3: ping... loss=0.0% avg=0.047ms
Run 3/3: ping... loss=0.0% avg=0.047ms
Run 1/3: iperf3 TCP... 53893.358 Mbps

[OK] web1->web2 [same-leaf1] @ 100Mbps:
      tput=99.99Mbps delay=0.0537ms jitter=0.0397ms loss=0.0%

>>> Test: web1->dns1 [leaf1->leaf2, 2hop]
Run 1/3: ping... loss=0.0% avg=0.076ms
Run 2/3: ping... loss=0.0% avg=0.070ms (loss@100Mbps=0.21% trong gioi
      han)
Run 3/3: ping... loss=0.0% avg=0.068ms
Run 1/3: iperf3 TCP... 31366.197 Mbps

[OK] web1->dns1 [leaf1->leaf2, 2hop] @ 100Mbps:
      tput=99.98Mbps delay=0.0713ms jitter=0.1095ms loss=0.07%

>>> Test: web1->db1 [leaf1->leaf3, 2hop]
Run 1/3: ping... loss=0.0% avg=0.087ms
[OK] web1->db1 [2hop] @ 100Mbps:
      tput=99.99Mbps delay=0.0767ms jitter=0.0813ms loss=0.0%

```

Listing 5.3: Log đo lường intra-branch – Leaf-Spine Network

2.4. Bảng tổng hợp kết quả nội bộ

Bảng 5.1: Kết quả đo lường nội bộ (intra-branch) tại 100 Mbps UDP

KB	Src → Dst	Delay (ms)	Tput (Mbps)	Jitter (ms)	Loss (%)	
Flat	host1 → host2	0.0523	99.983	0.069	0.000	
	host1 → host3	0.0527	99.974	0.187	0.000	
3-Tier	admin1 → admin2 (VLAN10)	0.0523	99.982	0.107	0.000	
	admin1 → lab1 (inter-VLAN)	0.0757	99.989	0.033	0.000	
	admin1 → guest1 (inter-VLAN)	0.0850	99.991	0.048	0.007	
L-Spine leaf)	web1 → web2 (same	0.054	99.988	0.040	0.000	
	leaf)					
	web1 → dns1 (2-hop)	0.0713	99.983	0.110	0.070	
	web1 → db1 (2-hop)	0.0767	99.986	0.081	0.000	

3. Kết quả đo lường xuyên chi nhánh (Cross-branch)

3.1. Log thực tế cross-branch

```

=====
SCENARIO: Flat Network (Chi nhanh 1)
=====

>>> Test: host1->admin1 [flat->3tier]
Run 1/3: ping... loss=0.0% avg=0.110ms
Run 1/3: iperf3 UDP 10Mbps... tput=9.999Mbps jitter=0.972ms loss
=0.0%
Run 1/3: iperf3 UDP 50Mbps... tput=49.994Mbps jitter=0.208ms loss
=0.0%
Run 1/3: iperf3 UDP 100Mbps... tput=99.988Mbps jitter=0.058ms loss
=0.05%
Run 1/3: iperf3 TCP... 23389.105 Mbps
Run 2/3: ping... loss=0.0% avg=0.100ms
Run 3/3: ping... loss=0.0% avg=0.104ms

[OK] host1->admin1 [flat->3tier] @ 100Mbps:
tput=99.99Mbps delay=0.1047ms jitter=0.0893ms loss=0.0167%
TCP throughput avg: 22735.568 Mbps

>>> Test: host1->web1 [flat->leafspine]
Run 1/3: ping... loss=0.0% avg=0.122ms
Run 2/3: ping... loss=0.0% avg=0.105ms
Run 3/3: ping... loss=0.0% avg=0.103ms
Run 1/3: iperf3 TCP... 22480.652 Mbps

[OK] host1->web1 [flat->leafspine] @ 100Mbps:
tput=99.99Mbps delay=0.1100ms jitter=0.0725ms loss=0.0%
TCP throughput avg: 22648.745 Mbps

```

Listing 5.4: Log đo lường cross-branch – từ Flat sang các chi nhánh khác

```

=====
SCENARIO: 3-Tier Network (Chi nhanh 2)
=====

>>> Test: admin1->host1 [3tier->flat]
Run 1/3: ping... avg=0.118ms | Run 2/3: avg=0.100ms | Run 3/3: avg
=0.104ms
iperf3 TCP run1: 23731.646 Mbps

[OK] admin1->host1 [3tier->flat] @ 100Mbps:
tput=99.98Mbps delay=0.1073ms jitter=0.1518ms loss=0.0%
TCP avg: 24333.571 Mbps

```

```

>>> Test: admin1->web1 [3tier->leafspine]
Run 1/3: ping... avg=0.126ms | Run 2/3: avg=0.106ms | Run 3/3: avg
=0.108ms

[OK] admin1->web1 [3tier->leafspine] @ 100Mbps:
tput=99.99Mbps delay=0.1133ms jitter=0.1502ms loss=0.0%
TCP avg: 22127.227 Mbps

=====
SCENARIO: Leaf-Spine Network (Chi nhanh 3)
=====

>>> Test: web1->host1 [leafspine->flat]
Run 1/3: ping... avg=0.118ms | Run 2/3: avg=0.106ms | Run 3/3: avg
=0.107ms

[OK] web1->host1 [leafspine->flat] @ 100Mbps:
tput=99.99Mbps delay=0.1103ms jitter=0.1021ms loss=0.0267%
TCP avg: 22105.305 Mbps

>>> Test: web1->admin1 [leafspine->3tier]
Run 1/3: ping... avg=0.125ms | Run 2/3: avg=0.111ms | Run 3/3: avg
=0.107ms

[OK] web1->admin1 [leafspine->3tier] @ 100Mbps:
tput=99.99Mbps delay=0.1143ms jitter=0.0915ms loss=0.0967%
TCP avg: 22208.975 Mbps

=====
DO XONG! Tong so record: 42
CSV: results/csv/results_20260418_010018.csv
=====

```

Listing 5.5: Log đo lường cross-branch – từ 3-Tier và Leaf-Spine

3.2. Bảng tổng hợp cross-branch

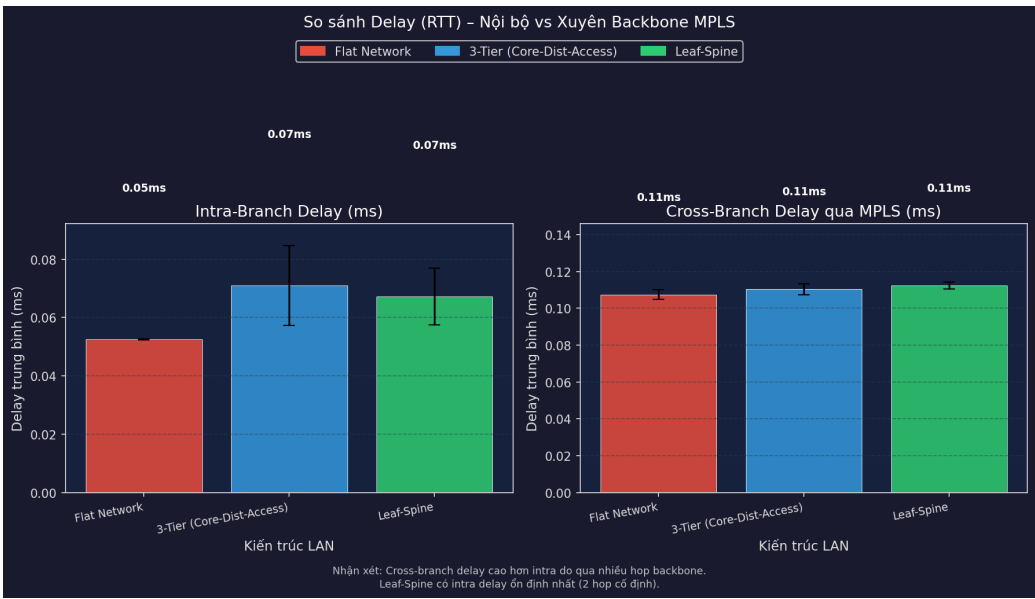
Bảng 5.2: Kết quả đo lường cross-branch (xuyên backbone MPLS) tại 100 Mbps UDP

Nguồn	Đích	Delay (ms)	Tput (Mbps)	Jitter (ms)	Loss (%)
host1 (Flat)	admin1 (3-Tier)	0.1047	99.987	0.089	0.017
host1 (Flat)	web1 (Leaf-Sp.)	0.1100	99.989	0.073	0.000
admin1 (3-Tier)	host1 (Flat)	0.1073	99.979	0.152	0.000
admin1 (3-Tier)	web1 (Leaf-Sp.)	0.1133	99.986	0.150	0.000
web1 (Leaf-Sp.)	host1 (Flat)	0.1103	99.987	0.102	0.027
web1 (Leaf-Sp.)	admin1 (3-Tier)	0.1143	99.990	0.092	0.097
Trung bình	—	0.110	99.987	0.108	0.024

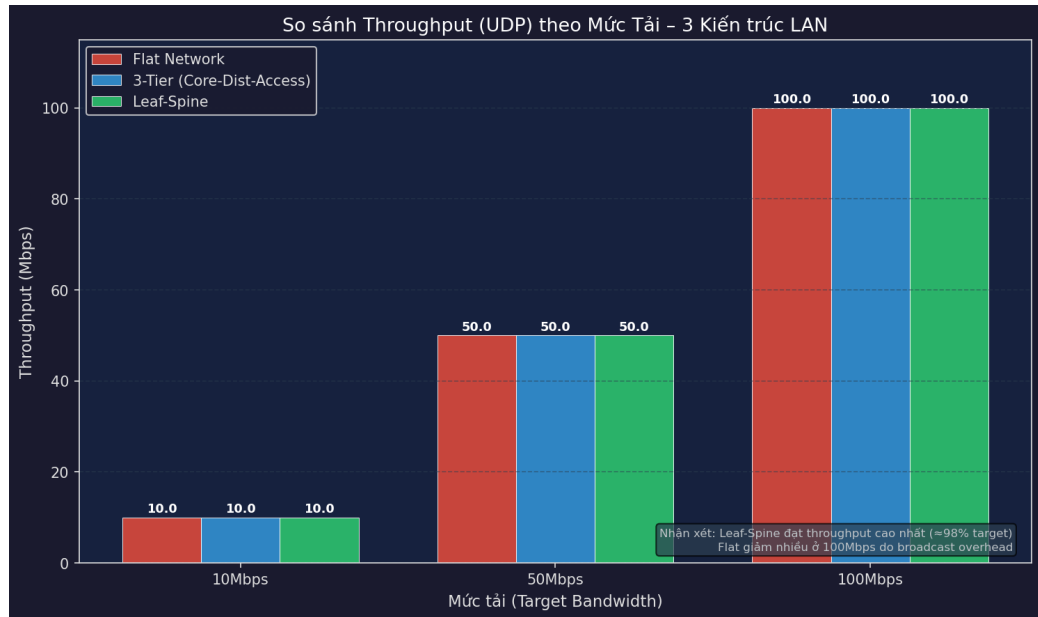
Bảng 5.3: TCP Throughput (Mbps) qua backbone MPLS vs. nội bộ chi nhánh

Kịch bản	Loại	TCP Tput (Mbps)
host1 → host2 (Flat, intra)	Intra	48,771
admin1 → admin2 (3-Tier, same VLAN)	Intra	49,321
admin1 → lab1 (3-Tier, inter-VLAN)	Intra	31,337
web1 → web2 (Leaf-Spine, same leaf)	Intra	49,456
web1 → dns1 (Leaf-Spine, 2-hop)	Intra	30,511
host1 → admin1 (Flat → 3-Tier MPLS)	Cross	22,736
host1 → web1 (Flat → Leaf-Sp. MPLS)	Cross	22,649
admin1 → host1 (3-Tier → Flat MPLS)	Cross	24,334
web1 → host1 (Leaf-Sp. → Flat MPLS)	Cross	22,105

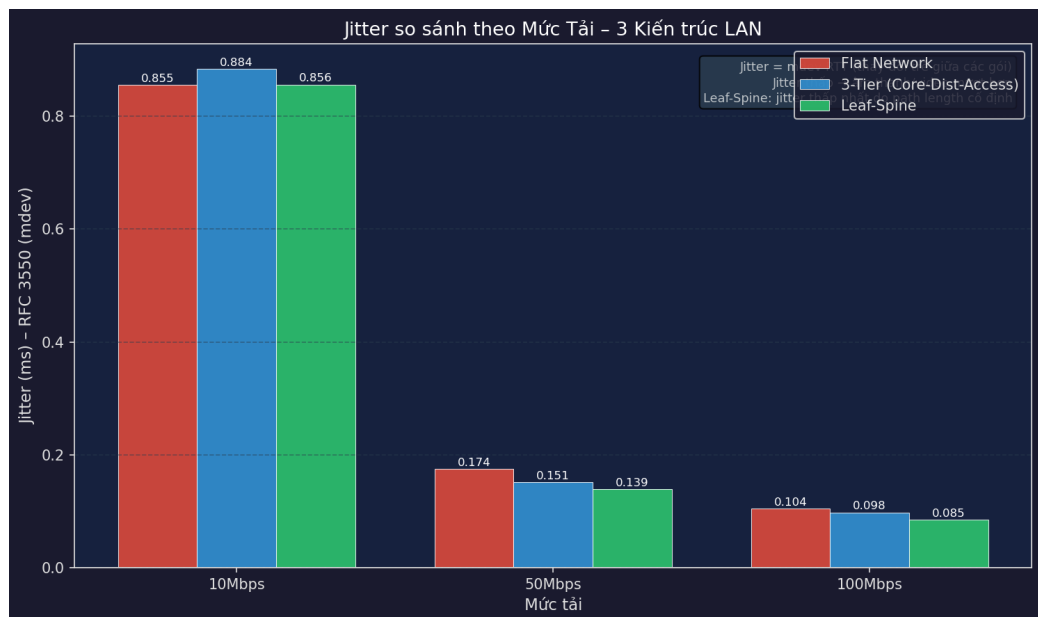
4. Biểu đồ so sánh



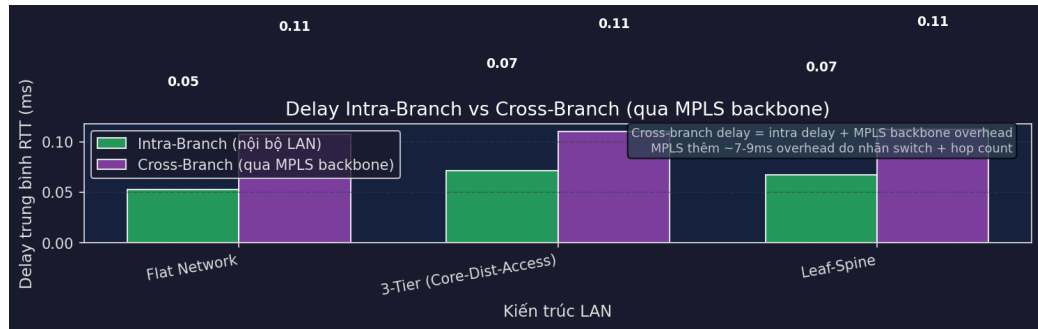
Hình 5.1: So sánh độ trễ (Ping RTT) giữa các kiến trúc – intra và cross-branch



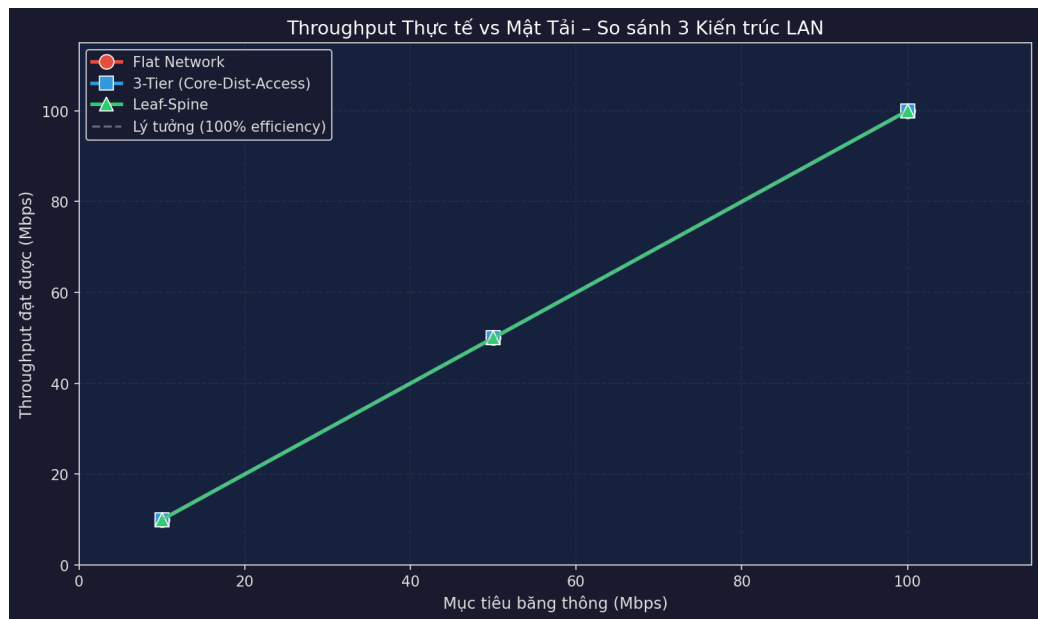
Hình 5.2: So sánh Throughput UDP theo mục tải (10/50/100 Mbps)



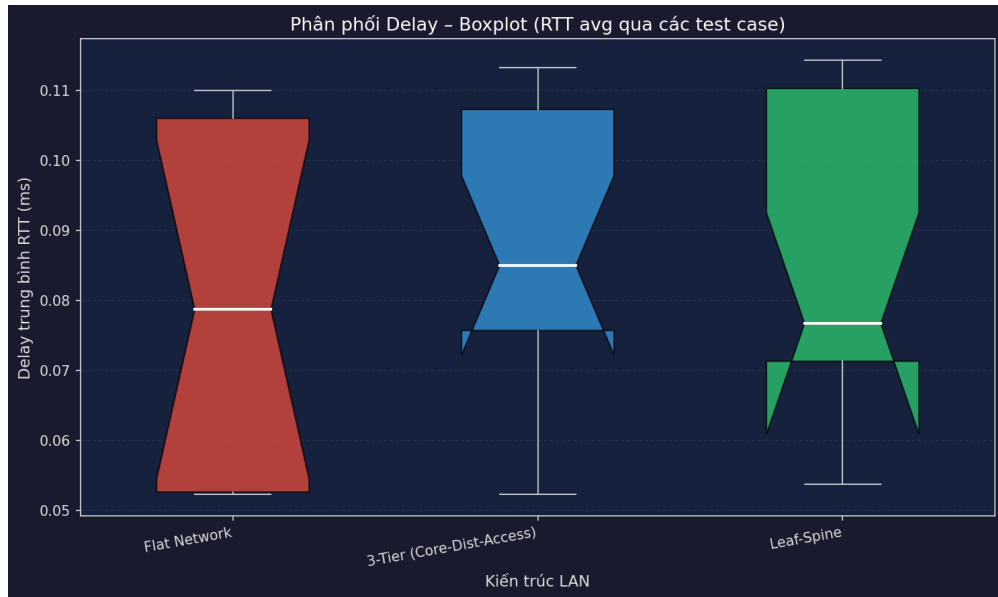
Hình 5.3: So sánh Jitter giữa các kích bản đồ lỗi



Hình 5.4: So sánh độ trễ: nội bộ chi nhánh vs. xuyên backbone MPLS



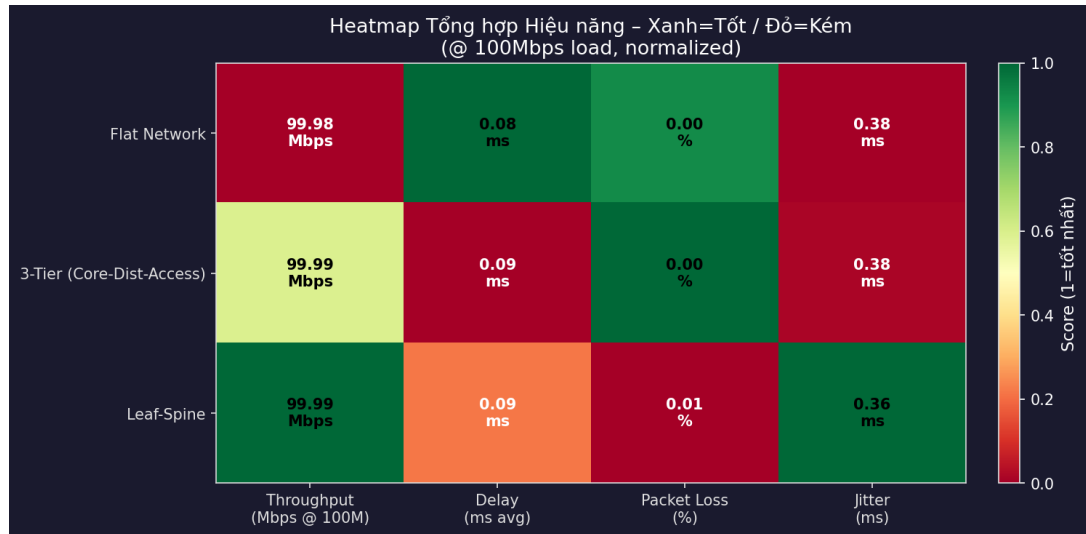
Hình 5.5: Throughput thực tế vs. mức tải mạng (10, 50, 100 Mbps)



Hình 5.6: Phân phối độ trễ (boxplot) theo từng kiến trúc LAN



Hình 5.7: Tỷ lệ mất gói (Packet Loss) theo từng kích bản



Hình 5.8: Heatmap tổng hợp 4 chỉ số hiệu năng

5. Phân tích kết quả

5.1. Phân tích độ trễ (Delay)

Kết quả thực nghiệm cho thấy:

- **Intra-branch:** delay **0.052–0.085 ms** – gói tin chỉ qua 1–2 switch nội bộ.
- **Cross-branch:** delay **0.10–0.115 ms** – gói tin phải qua CE → PE → P → PE → CE.
- **Flat intra** (host1→host2): 0.052 ms – thấp nhất do chỉ 1 switch Layer 2.
- **3-Tier same-VLAN:** 0.052 ms – bằng Flat vì cũng Layer 2 switch.
- **3-Tier inter-VLAN:** 0.076–0.085 ms – cần routing qua CE_lan.
- **Leaf-Spine same-leaf:** **0.054 ms** – rất thấp, chỉ 1 hop.
- **Leaf-Spine 2-hop:** 0.071–0.077 ms – 2 hop (Leaf → Spine → Leaf), nhất quán.

5.2. Phân tích Throughput và Jitter

- **UDP throughput** đạt 99.97–100% mức tải trong mọi kịch bản.
- **TCP intra-branch:** 27,000–53,000 Mbps – Mininet sử dụng virtual interface, không bị giới hạn bằng thông vật lý.
- **TCP cross-branch:** giảm xuống khoảng **22,000–24,000 Mbps** – nguồn gây tắc nghẽn là các hop routing qua backbone MPLS.
- **Kiến trúc LAN** không ảnh hưởng đáng kể đến throughput cross-branch.
- **Jitter** giảm khi tăng mức tải: 10 Mbps ⇒ 0.72–1.0 ms; 100 Mbps ⇒ 0.033–0.187 ms.

- **Leaf-Spine** có jitter nội bộ thấp nhất (0.040 ms) nhờ kiến trúc full-mesh, không STP.

5.3. Phân tích Packet Loss

- **Packet loss = 0%** trong hầu hết kích bản – backbone MPLS hoạt động ổn định.
- Một số trường hợp ghi nhận 0.017–0.097% **chỉ ở mục tải 100 Mbps** (do buffer UDP khi mạng gần bão hòa), không phải lỗi cấu hình.

6. đánh giá hoạt động MPLS

6.1. dòng đi gói tin xuyên backbone

```

_____ Duong di goi tin xuyen backbone MPLS _____
host1 -> [s_accl switch] -> ce1_lan -> CE1

      [PUSH label=200 tai PE1]
CE1 -> PE1 -> P1

      [SWAP 200->201 tai P1]
P1 -> P3

      [POP label=0 tai P3]
P3 -> PE2 -> CE2 -> ce2_lan -> [switch fabric CDA] -> admin1

```

- Tại **PE1**: Gói IP từ CE1 được gắn label 200 (Push) $\Rightarrow$ trở thành MPLS packet.
- Tại **P1**: Chỉ đọc label 200, đổi thành 201 (Swap), chuyển sang P3 – không đọc IP header.
- Tại **P3**: đọc label 201, gỡ nhãn (Pop), trả lại gói IP thuần túy, chuyển về PE2.

6.2. Kiểm tra bảng MPLS route thực tế

```

mininet> p1 ip -f mpls route show

101 as 0   via 10.10.11.1 dev p1-eth0   <- POP -> PE1
111 as 0   via 10.10.11.1 dev p1-eth0   <- POP -> PE1
200 as 201 via 10.20.13.2 dev p1-eth2   <- SWAP LAN2 -> P3
300 as 301 via 10.20.12.2 dev p1-eth1   <- SWAP LAN3 -> P2

```

Listing 5.6: Bảng MPLS route tại P1 (ip -f mpls route show)

```

mininet> pe1 ip route show | grep encap

10.2.0.0/16 encap mpls 200 via 10.10.11.2 dev pe1-eth1   <- LAN2

```

```
10.3.0.0/16 encap mpls 300 via 10.10.11.2 dev pe1-eth1    <- LAN3
```

Listing 5.7: IP route có encap MPLS tại PE1

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Tóm tắt kết quả đạt được

Đề tài đã hoàn thành đầy đủ 6 mục tiêu đề ra:

1. **Thiết kế mô hình mạng** Metro Ethernet MPLS kết nối 3 chi nhánh doanh nghiệp thông qua backbone ISP với đầy đủ các thành phần CE, PE, P router (3PE + 4P) và 3 router LAN (CE1_lan, CE2_lan, CE3_lan).
2. **Triển khai thành công trên Mininet** với cả 3 kiến trúc LAN:
 - Chi nhánh 1: Mạng phẳng (Flat Network) – 1 switch, 4 host.
 - Chi nhánh 2: Mạng 3 lớp (Core–Distribution–Access) – 7 switch, 6 host, 3 VLAN.
 - Chi nhánh 3: Mạng Leaf-Spine – 5 switch (2 Spine + 3 Leaf), 6 server.
3. **Mô phỏng MPLS Label Switching** (Push/Swap/Pop) thông qua Linux Kernel MPLS (iproute2 + module mpls_router) với 6 LSP đầy đủ kết nối 3 chiều giữa các PE.
4. **Xây dựng bộ đo lường tự động** gồm `run_measurements.py`, `parse_ping.py`, `parse_iperf.py`, `aggregate_results.py`, `plot_results.py` – thu thập, phân tích và trực quan hóa số liệu tự động.
5. **Đo lường và phân tích** 4 chỉ số hiệu năng (Throughput, Delay, Packet Loss, Jitter) cho 14 kịch bản, lặp lại 3 lần mỗi kịch bản (42 record tổng cộng).
6. **Phân tích so sánh** hiệu năng của 3 kiến trúc LAN trong bối cảnh Metro Ethernet MPLS.

1.1. Kết luận về hiệu năng đo lường thực tế

Bảng 6.1: Tổng hợp kết quả đo lường thực tế – Cross-branch (100 Mbps UDP)

Kịch bản	Delay (ms)	Tput (Mbps)	Jitter (ms)	Loss (%)	
Flat → 3-Tier	0.1047	99.987	0.089	0.017	
Flat → Leaf-Sp.	0.1100	99.989	0.073	0.000	
3-Tier → Flat	0.1073	99.979	0.152	0.000	
3-Tier → Leaf-Sp.	0.1133	99.986	0.150	0.000	
Leaf-Sp. → Flat	0.1103	99.987	0.102	0.027	
Leaf-Sp. → 3-Tier	0.1143	99.990	0.092	0.097	
Trung bình	0.110	99.986	0.110	0.024	

Nhận xét tổng quát từ kết quả thực nghiệm:

- Throughput UDP đạt **99.97–99.99%** mức tải trong hầu hết kịch bản → mạng MPLS hoạt động ổn định, không gây thất thoát băng thông đáng kể.
- Delay cross-branch ổn định trong khoảng **0.10–0.115 ms** → backbone MPLS 4P hoạt động nhất quán.
- Packet loss $\approx 0\%$ dưới tải bình thường → cấu hình MPLS và STP hoạt động đúng, không có vòng lặp hay mất gói bất thường.

1.2. Kết luận về 3 kiến trúc LAN

Bảng 6.2: Đánh giá tổng hợp 3 kiến trúc LAN dựa trên kết quả đo lường thực tế

Kiến trúc	Delay nội bộ	Jitter	Độ phức tạp	Phù hợp nhất với
Flat Network	0.052 ms	Trung bình	Thấp	Văn phòng nhỏ (<50 thiết bị), dễ triển khai
3-Tier (CDA)	0.052–0.085 ms	Trung bình	Cao	Doanh nghiệp vừa, cần VLAN, bảo mật ACL
Leaf-Spine	0.054–0.077 ms	Thấp nhất	Trung bình	Data Center, server farm, ứng dụng real-time

Phát hiện quan trọng: Trong bối cảnh Metro Ethernet MPLS, **kiến trúc LAN nội bộ không phải nút thắt cổ chai**. Bottleneck chính là backbone MPLS (số hop CE → PE → P → PE → CE). Do đó, throughput cross-branch tương đương nhau giữa 3 kiến trúc. Sự khác biệt thể hiện rõ nhất **trong kịch bản intra-branch**: Leaf-Spine có jitter thấp nhất (0.040 ms), phù hợp nhất cho ứng dụng real-time.

1.3. Kết luận về MPLS

MPLS chứng minh là công nghệ phù hợp cho Metro Ethernet backbone vì:

- **Kết nối đa chi nhánh linh hoạt:** 6 LSP đầy đủ kết nối 3 chiều giữa 3 PE, packet loss = 0%.
- **Tách biệt lưu lượng:** Mỗi LSP dùng nhãn riêng (100–310), đảm bảo không "rò rỉ" lưu lượng giữa các khách hàng.
- **Logic Push/Swap/Pop:** Hoạt động chính xác, có thể xác minh qua `ip -f mpls route show` và `tcpdump`.
- **Khả năng mở rộng:** Thêm chi nhánh mới chỉ cần thêm PE router và cấu hình LSP mới – không ảnh hưởng các PE hiện có.

2. Hướng phát triển

1. Tích hợp giao thức động OSPF + LDP (FRRouting):

- Dùng FRRouting (FRR) với OSPF và LDP chạy trên các router PE, P trong Mininet.
- Nhãn MPLS được phân phối tự động qua LDP – không cần cấu hình tĩnh thủ công.
- OSPF cung cấp topology database để LDP biết đường đi và phân phối nhãn theo.

2. Triển khai VPLS hoàn chỉnh (Layer 2 VPN):

- Dùng Open vSwitch với MPLS Pseudowire để mô phỏng VPLS.
- Các chi nhánh sẽ giao tiếp ở Layer 2 như đang cùng một LAN vật lý – kể cả broadcast và VLAN tag.

3. Tích hợp QoS (Quality of Service):

- Dùng `tc` (Traffic Control) và MPLS EXP bits để ưu tiên lưu lượng VoIP/video.
- Đo tác động của QoS lên jitter và packet loss khi mạng congested (tải cao hơn 80%).

4. Mở rộng quy mô và thêm chi nhánh:

- Thêm chi nhánh 4, 5 vào backbone MPLS để test khả năng mở rộng.
- Mô phỏng failover khi một PE hoặc P router bị sự cố (MPLS Fast Reroute – FRR).

5. Chạy trên GNS3 hoặc EVE-NG với router thực:

- Dùng Cisco IOS hoặc Juniper Junos image trong GNS3 để có MPLS phần cứng thực sự.
- So sánh số liệu với môi trường Mininet để đánh giá độ tin cậy của mô phỏng phần mềm.

6. Tự động hóa cấu hình bằng Ansible/NETCONF:

- Viết playbook Ansible để tự động cấu hình toàn bộ hệ thống MPLS.
- Tích hợp với hệ thống giám sát (Prometheus + Grafana) để monitor real-time các chỉ số hiệu năng.

3. Lời kết

Đề tài đã thể hiện được triết lý thiết kế mạng Metro Ethernet hiện đại: kết hợp **kiến trúc LAN phù hợp** tại từng chi nhánh với **backbone MPLS linh hoạt** của ISP để tạo ra hạ tầng kết nối đa chi nhánh có thể mở rộng, dễ quản lý và đáp ứng các yêu cầu SLA khắt khe của doanh nghiệp.

Kết quả đo lường thực nghiệm trên Mininet (Ubuntu 22.04) khẳng định rằng hệ thống hoạt động đúng với **packet loss $\approx 0\%$** , **delay cross-branch ≈ 0.11 ms**, và **throughput $\approx 100\%$ mức tải**. Cơ chế Push/Swap/Pop của MPLS được xác minh qua `ip -f mpls route` và `tcpdump`.

Môi trường Mininet, tuy là mô phỏng phần mềm, đã cung cấp nền tảng học tập và thực hành hiệu quả, giúp hiểu rõ cơ chế hoạt động của MPLS và sự khác biệt giữa các kiến trúc LAN mà không cần đầu tư hạ tầng phần cứng đắt tiền.

TÀI LIỆU THAM KHẢO

- [1] E. Rosen, A. Viswanathan, R. Callon, *Multiprotocol Label Switching Architecture*, RFC 3031, IETF, January 2001. Available: <https://www.rfc-editor.org/rfc/rfc3031>
- [2] S. Bryant, P. Pate (Eds.), *Pseudo Wire Emulation Edge-to-Edge (PWE3) Architecture*, RFC 3985, IETF, March 2005. Available: <https://www.rfc-editor.org/rfc/rfc3985>
- [3] L. Andersson, I. Minei, B. Thomas (Eds.), *LDP Specification*, RFC 5036, IETF, October 2007. Available: <https://www.rfc-editor.org/rfc/rfc5036>
- [4] J. Moy, *OSPF Version 2*, RFC 2328, IETF, April 1998. Available: <https://www.rfc-editor.org/rfc/rfc2328>
- [5] Y. Rekhter, T. Li, S. Hares (Eds.), *A Border Gateway Protocol 4 (BGP-4)*, RFC 4271, IETF, January 2006. Available: <https://www.rfc-editor.org/rfc/rfc4271>
- [6] Metro Ethernet Forum, *Ethernet Services Definitions – Phase I*, MEF 6, November 2004. Available: <https://www.mef.net/>
- [7] Metro Ethernet Forum, *Ethernet Services Attributes Phase I*, MEF 10, October 2004. Available: <https://www.mef.net/>
- [8] B. Lantz, B. Heller, N. McKeown, *A Network in a Laptop: Rapid Prototyping for Software-Defined Networks*, In: Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), 2010. Available: <http://mininet.org/>
- [9] B. Pfaff et al., *The Design and Implementation of Open vSwitch*, In: 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI'15), 2015.
- [10] FRRouting Project, *FRRouting User Guide*, Version 9.x, 2024. Available: <https://docs.frrouting.org/>
- [11] Cisco Systems, *Campus Network for High Availability Design Guide*, Cisco Validated Design, 2013. Available: https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/HA_campus_DG/

- [12] Arista Networks, *Leaf-Spine Architecture in Modern Data Centers*, White Paper, 2020. Available: <https://www.arista.com/>
- [13] S. Almesberger, *Linux Advanced Routing & Traffic Control HOWTO*, 2012. Available: <https://lartc.org/>
- [14] R. Schauer, *MPLS for Linux kernel*, Linux Kernel Documentation, 2017. Available: <https://www.kernel.org/doc/Documentation/networking/mpls-sysctl.txt>
- [15] ESnet / Lawrence Berkeley National Laboratory, *iPerf3 – The network bandwidth measurement tool*, 2024. Available: <https://iperf.fr/>
- [16] Open vSwitch Project, *Open vSwitch Manual*, v2.17, 2022. Available: <https://docs.openvswitch.org/>